

# **İşletim Sistemleri Bellek Yönetimi**

---

Dr. Öğr. Üyesi Ertan Bütün

Bu dersin içeriği hazırlanırken Operating System Concepts (Silberschatz, Galvin and Gagne) kitabı ve Prof. Dr. M. Ali Akcayol'un (Gazi Üniversitesi Bilgisayar Mühendisliği Bölümü) ders sunumlarından faydalanılmıştır.

<https://codex.cs.yale.edu/avi/os-book/OS9/slide-dir/index.html>

<http://w3.gazi.edu.tr/~akcayol/BMOS.htm>

# Konular

- Giriş
- Swapping
- Bitişik Bellek tahsisi
- Segmentation
- Paging

# Giriş

- Modern bilgisayar sistemlerinin çalışmasında bellek merkezi role sahiptir.
- Bellek, her birisi kendi adresine sahip olan çok sayıda bayt alanına sahiptir.
- Tipik bir komut yürütme döngüsü (instruction execution cycle) şöyledir:
  - CPU, program counter değerine göre bellekten bir komutu getirir (**fetch**).
  - Komut çözülür (**decode**) ve komutun yürütülmesi için işlenecek verilerin (operand) bellekten alınması gerekiyorsa gerekli veriler bellekten getirilir.
  - Komut ilgili veriler üzerinde işlemini tamamlar (**execute**), sonuçlar belleğe geri kaydedilebilir.
- Bellek birimi ise yalnızca bir bellek adresleri akışını görür; nasıl üretildiklerini veya ne için olduklarını (komut mu veya veri mi) bilmez.
  - Yalnızca çalışan program tarafından oluşturulan bellek adreslerinin sırası ile ilgilenilir.

# Giriş - Temel donanım yapısı

- Ana bellek (**main memory**) ve işlemcide yerleşik olan yazmaçlar (**registers**), CPU'nun **doğrudan erişebildiği tek genel amaçlı depolama birimleridir**.
- Makine komutlarında bellek adresini parametre (operand) olarak alan komutlar vardır, ancak disk adresini alan komutlar yoktur.
- Bu nedenle, yürütülmekte olan tüm komutlar ve kullanılan tüm veriler bu **doğrudan erişimli depolama cihazlarından birinde olmalıdır** (ana bellek, yazmaç).
- Veriler bellekte değilse, CPU'nun bunlar üzerinde işlem yapabilmesi için önce belleğe alınmalıdır.

# Giriş - Temel donanım yapısı

- CPU içerisindeki register'lara genellikle CPU saatinin bir cycle ile erişilebilmektedir.
- Aynı şey, bellek veri yolu (bus) ile erişilen ana bellek için söylenemez. Bir bellek erişiminin tamamlanması, CPU saatinin birçok cycle'ını alabilir.
  - Bu tür durumlarda, işlemcinin, yürütmekte olduğu komutu tamamlamak için gereken verilere sahip olmadığından normal olarak durması (**stall**) gerekir.
  - Bu durum, bellek erişimlerinin sıklığı nedeniyle tahammül edilemez.
  - Çözüm, hızlı erişim için CPU yongasına önbellek (**cache**) eklemektir.

# Bellek Alanlarının Korunması

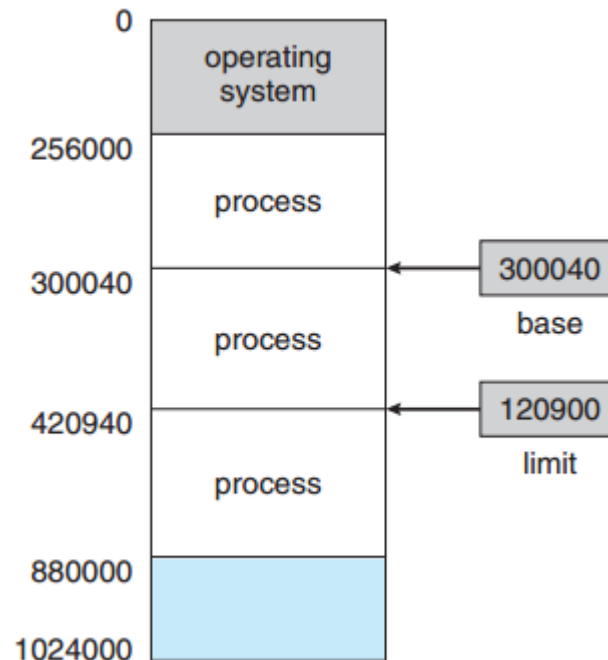
- Bellek yönetiminde fiziksel belleğe erişim hızının yanı sıra bellek üzerinden yürütülen process'lerin doğru çalışması da ele alınması gereken bir konudur.
- Sistemin düzgün çalışması için işletim sisteminin kullandığı bellek alanları kullanıcı process'lerin erişiminden korunmalıdır.
- Çok kullanıcıli sistemlerde, ek olarak kullanıcı process'lerinin kullandığı bellek alanları birbirinden korunmalıdır.
- Bu koruma performansın düşmemesi için donanım tarafından sağlanmalıdır. İşletim sistemi genellikle CPU ve CPU'nun bellek erişimleri arasına müdahale etmez.

# Donanım ile Bellek Alanlarının Korunması

- Donanım ile bellek alanlarının korunması farklı yollarla gerçekleştirilebilir. Şimdi örnek bir gerçekleştirim verilecektir:
  - Öncelikle her process'in ayrı bir bellek alanına sahip olduğundan emin olmalıyız.
  - Process başına ayrı bellek alanı, process'leri birbirinden korur ve eşzamanlı yürütme için birden çok process'in belleğe yüklenmesi için temel bir gereksinimdir.
  - Bellek alanlarını ayırmak için, process'in erişebileceği legal adres aralığını belirlemeye ve process'in yalnızca bu legal adreslere erişebilmesini sağlamaya ihtiyacımız var.

# Donanım ile Bellek Alanlarının Korunması

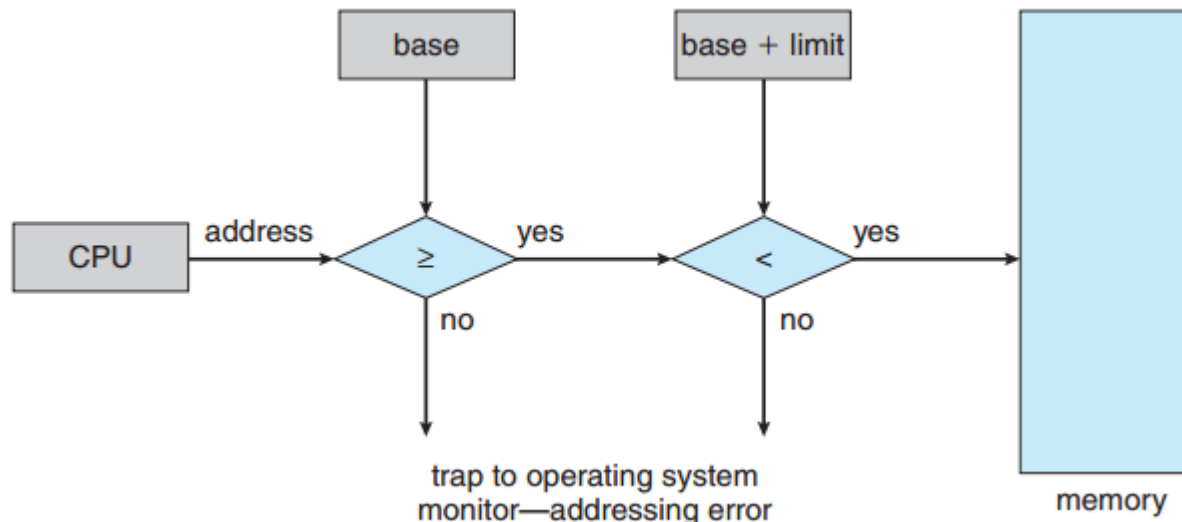
- Bu korumayı şekilde gösterildiği gibi, genellikle bir **base** register ve bir **limit** register olmak üzere iki **register** kullanarak sağlayabiliriz:
  - **Base register**, legal fiziksel bellek adresinin başlangıç değerini tutar.
  - **limit register**, aralığın boyutunu belirtir.





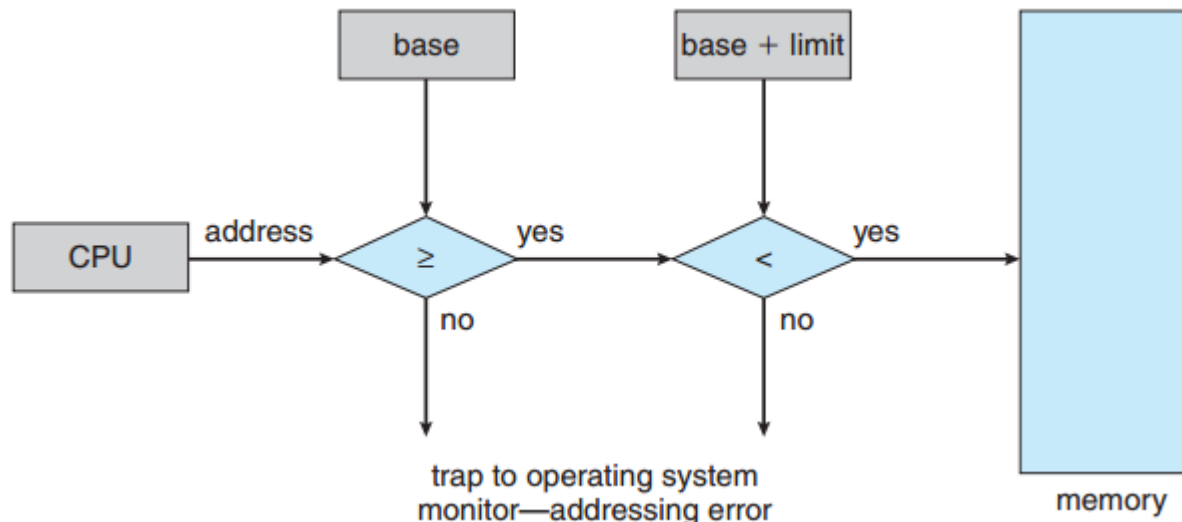
# Donanım ile Bellek Alanlarının Korunması

- Bellek alanının korunması, CPU donanımının kullanıcı modunda üretilen her adresi yazmaçlarla karşılaştırmasıyla gerçekleştirilir.
- Şekildeki şema, bir kullanıcı programının işletim sisteminin veya diğer kullanıcıların kodunu veya veri yapılarını (yanlışlıkla veya kasıtlı olarak) değiştirmesini önler.
- Kullanıcı modunda çalıştırılan bir programın işletim sistemi belleğine veya diğer kullanıcıların belleğine erişme girişimleri, işletim sisteminde bir hataya neden olur ve bu girişim ölümcül bir hata olarak değerlendirilir.



# Donanım ile Bellek Alanlarının Korunması

- Base ve limit register'leri yalnızca özel bir öncelikli komut ile işletim sistemi tarafından yüklenebilir.
- Öncelikli komutlar yalnızca çekirdek modunda yürütülebildiğinden ve yalnızca işletim sistemi çekirdek modunda yürütüldüğünden, base ve limit register'lerini yalnızca işletim sistemi yükleyebilir.
- Bu şema, işletim sisteminin register'lerin değerini değiştirmesine izin verirken kullanıcı programların register'lerin içeriğini değiştirmesini engeller.

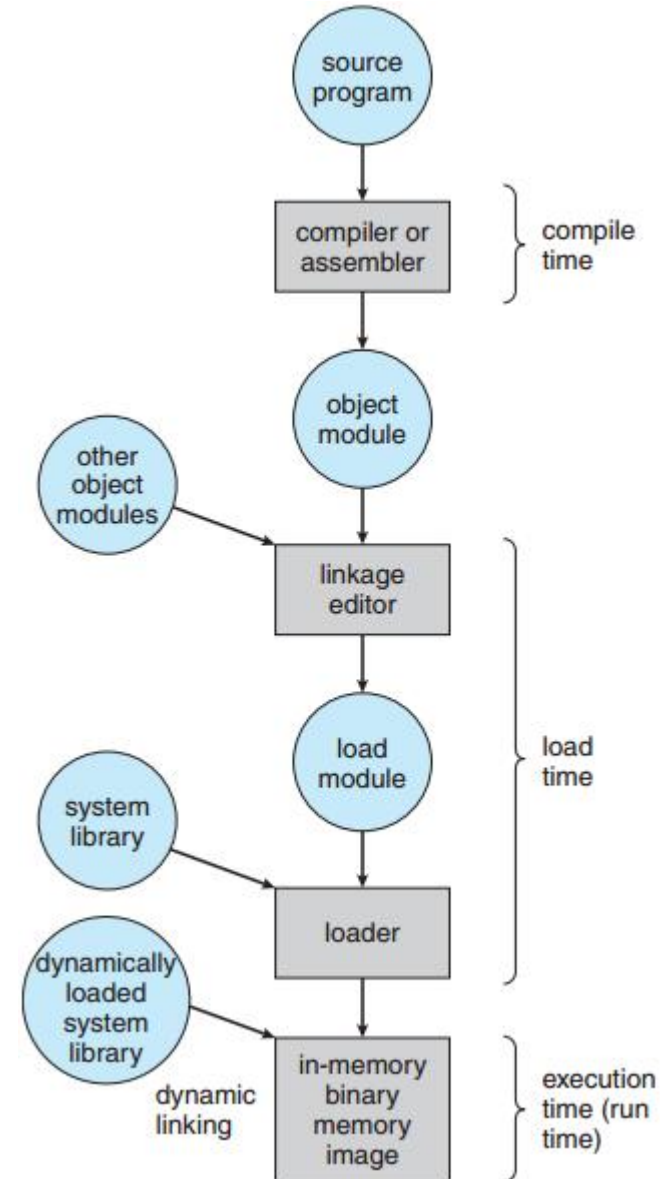


# Adres bağlama (binding)

- Bir program disk üzerinde binary dosya olarak bulunur.
- **Bir programın çalıştırılabilmesi için belleğe alınması gereklidir.**
- Diskte yer alan ve yürütülmek üzere belleğe getirilmeyi bekleyen process'ler girdi kuyruğunu (**input queue**) oluşturur.
- Çoğu sistem, bir kullanıcı process'inin fiziksel belleğin herhangi bir bölümünde bulunmasına izin verir.
  - Bu nedenle, bilgisayarın adres alanı 00000'den başlasa da, kullanıcı process'inin ilk adresinin 00000 olması gerekmez.

# Derleme Süreci ve Linker

- Şekilde gösterildiği gibi çoğu durumda bir kullanıcı programı çalıştırılmadan önce bazıları isteğe bağlı olabilen birkaç adımdan geçer.
- Bu adımlar sırasında adresler farklı şekillerde gösterilebilir.
- Kaynak programdaki adresler genellikle semboliktir (count değişkeni gibi).
- Bir derleyici tipik olarak bu sembolik adresleri yeniden yerleştirilebilir adreslere (relocatable addresses) bağlar (**bind**). (örneğin modülün başından itibaren 14. bayt gibi).
- Bağlantı düzenleyici (linkage editor) veya yükleyici (loader) sırayla yeniden yerleştirilebilir adresleri mutlak adreslere (74014 gibi) bağlar.
- Her bağlama (**binding**), bir adres alanından diğerine yapılan eşlemedir.



# Adres bağlama (binding)

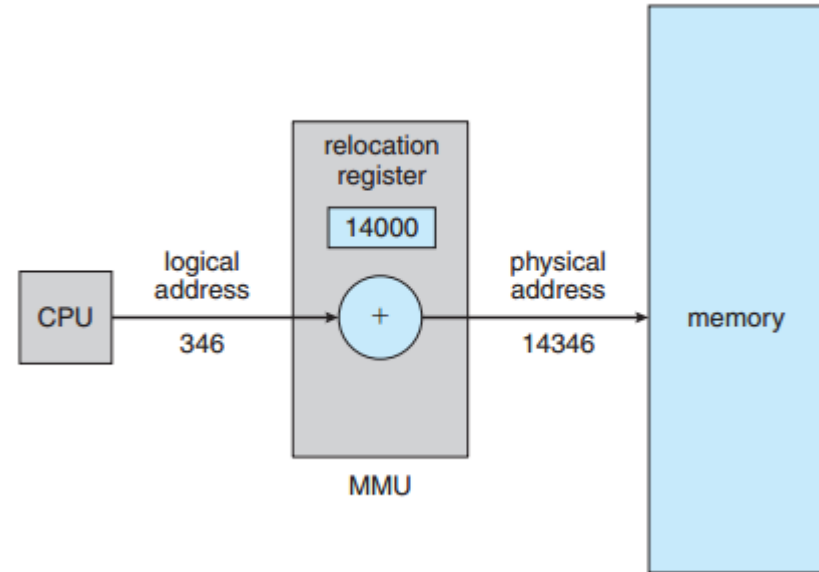
- Komutların ve verilerin bellek adreslerine **bağlanması (binding)** farklı şekillerde olabilir:
  - **Derleme zamanı:** Derleme aşamasında kodun bellekte yerleşeceği yer biliniyorsa mutlak kod (**absolute code**) oluşturulabilir.
    - Yerleşeceği bellek alanı değişirse yeniden derleme gereklidir.
  - **Yüklenme zamanı:** Derleme aşamasında programın yerleşeceği yer bilinmiyorsa, derleyici yeniden yerleştirilebilir kod (**relocatable code**) oluşturur.
    - Bu durumda, son bağlama (**binding**) yükleme zamanına kadar ertelenir.
    - Başlangıç adresi değişirse, bu değiştirilen değeri dahil etmek için yalnızca kullanıcı kodunu yeniden yüklememiz gerekir.
  - **Yürütme zamanı:** Process yürütülürken bir bellek bölümünden diğerine taşınabilirse, bağlama (**binding**) çalışma zamanına kadar ertelenmelidir.
    - Bu şemanın çalışması için özel donanım mevcut olmalıdır.
    - **Çoğu genel amaçlı işletim sistemi bu yöntemi kullanır.**

# Mantıksal ve Fiziksel Adres Alanı

- CPU tarafından oluşturulan adres, mantıksal adres (**logical address**) olarak adlandırılır, gerçek bir fiziksel adrese atıfta bulunmaz, sanal adres (**virtual address**) olarak da adlandırılır.
- Bellek biriminin gördüğü adres ise fiziksel adres (**physical address**) olarak adlandırılır. Mantıksal adresler bellek yönetim birimi (MMU) tarafından fiziksel bir adreslere çevrilir.
- Derleme zamanı ve yükleme zamanı adres bağlama yöntemleri aynı mantıksal ve fiziksel adresleri üretir.
- Yürütme zamanı adres bağlama şeması, **farklı** mantıksal ve fiziksel adreslerle sonuçlanır. Bu durumda, genellikle mantıksal adrese sanal adres de denir.
- Bir program tarafından üretilen tüm mantıksal adresler kümesi, mantıksal bir adres alanıdır (**logical address space**). Bu mantıksal adreslere karşılık gelen tüm fiziksel adresler kümesi, fiziksel bir adres alanıdır (**physical address space**) .

# Bellek Yönetimi Birimi

- Yürütme zamanında sanal adresin fiziksel adrese dönüştürülmesi **bellek yönetimi birimi** (**memory-management-unit: MMU**) olarak adlandırılan bir donanım bileşeni tarafından yapılır.
- Dönüştürme için birçok farklı yöntem vardır. Şimdilik, bu dönüştürme, base register şemasının bir genellemesi olan basit bir MMU şemasıyla gösterilecektir:
  - **Base register** bir yeniden yerleştirme yazmacı (**relocation register**) olarak adlandırılır.
  - Şekilde gösterildiği gibi adres belleğe gönderilirken **relocation register**'indeki değer, bir kullanıcı process'i tarafından üretilen her adrese eklenir.



# Dinamik yükleme

- Daha iyi bellek alanı kullanımı elde etmek için dinamik yüklemeyi (**dynamic loading**) kullanabiliriz:
  - Dinamik yüklemede, bir yordam (program parçası, **routine**) çağrılana kadar yüklenmez. Tüm yordamlar yeniden yüklenebilir bir formatta diskte tutulur.
  - Önce main program belleğe yüklenir ve çalıştırılır.
  - Bir program parçası (**routine**) çalışırken başka yordamı çağırdığında, bellekte yüklü değilse **loader** tarafından yüklenir.
- Dinamik yükleme, çok büyük boyuttaki programların çalıştırılması için bellek yönetimi açısından fayda sağlar.
- Dinamik yükleme, işletim sisteminden özel destek gerektirmez. Programlarını böyle bir yöntemden yararlanacak şekilde tasarlamak kullanıcıların sorumluluğundadır.
  - İşletim sistemleri, dinamik yüklemeyi uygulamak için kütüphane yordamları sağlayarak programcıya yardımcı olabilir.



# Dinamik Bağlama ve Paylaşılan Kütüphaneler

- Dinamik bağlanan kütüphaneler (**Dynamically linked libraries, DLL**), sistem kütüphaneleridir ve kullanıcı programına çalışırken bağlanırlar.
- Bazı işletim sistemleri yalnızca statik bağlamayı (**static linking**) destekler.
  - Sistem kütüphaneleri diğer herhangi bir object modülü gibi ele alınır, yükleyici tarafından binary formattaki programın imaj dosyasına statik bağlanır.
- Dinamik bağlamayla, her kütüphane yordamı referansı için program imaj dosyasına bir kod parçası (**stub**) eklenir.
  - **Stub**, bellekte yerleşik kütüphane yordamının nasıl bulunacağını veya yordam bellekte değilse kütüphanenin nasıl yükleneceğini gösteren küçük bir kod parçasıdır.

# Dinamik Bağlama ve Paylaşılan Kütüphaneler

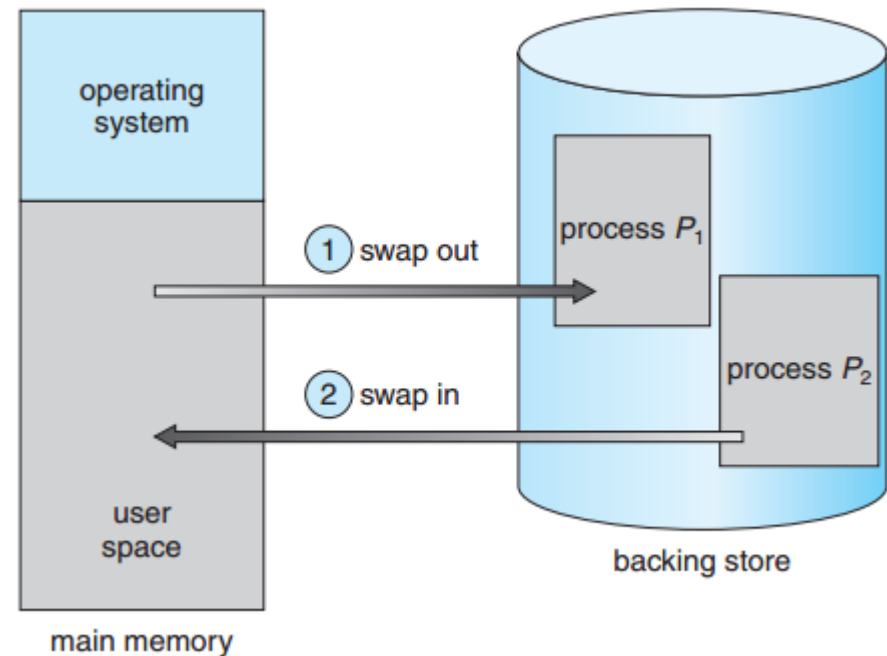
- **Stub** yürütüldüğünde, gerekli yordamın halihazırda bellekte olup olmadığını kontrol eder. Bellekte yoksa, program yordamı belleğe yükler.
- **Stub**, yordamın adresiyle kendini değiştirir ve yordamı yürütür. Böylelikle, belirli bir kod parçasına bir dahaki sefere ulaşıldığında, kütüphane yordamı doğrudan yürütülür ve dinamik bağlantı için herhangi bir maliyete neden olmaz.
- Bu şema altında, bir dilin kütüphanesini kullanan tüm process'ler kütüphane kodunun yalnızca bir kopyasını yürütür.
- Dinamik bağlama özellikle paylaşılan kütüphanelerde kullanışlıdır.

# Konular

- Giriş
- **Swapping**
- Bitişik bellek tahsisi
- Segmentation
- Paging

# Yer deęiřtirme (Swapping)

- Bir process alıřmak iin bellekte olmak zorundadır.
- Bir process geici olarak bellekten diske (**backing store**) yer deęiřtirilebilir (**swapped**) ve daha sonra yrtmeye devam etmek iin tekrar belleęe alınabilir.
- Yer deęiřtirme (**swapping**) , tm process'lerin toplam ihtiya duyduęu fiziksel adres alanının sistemin gerek fiziksel belleęini ařmasını mmkn kılar.
  - Bylece bir sistemdeki multiprocessing kabiliyeti artar.



# Yer deęiřtirme (Swapping)

- Yedekleme deposu (**backing store**) – genellikle hızlı bir disktir. Tüm kullanıcılar için tüm bellek görüntülerinin (image) kopyalarını barındıracak kadar büyük olmalı ve bu bellek görüntülerine doğrudan erişim sağlamalıdır.
- **Roll out, roll in** – öncelięe dayalı (priority-based) scheduling algoritmaları için kullanılan swapping biçimidir; düşük öncelikli process bellekten diske yer deęiřtirilir, böylece daha yüksek öncelikli process yüklenebilir ve yürütülebilir.
- Swapping süresinin büyük kısmı transfer süresidir. Toplam aktarım süresi, swap edilen bellek miktarı ile doğru orantılıdır.
- Sistem; bellek görüntüleri diskte veya bellekte olan çalışmaya hazır tüm process'leri bir hazır kuyruęunda (**ready queue**) tutar.

# Yer deęiřtirme (Swapping)

- Bellekten diske swap edilen process'in aynı fiziksel adreslere geri dönmesi gereklilięi, adres baęlama (binding) yöntemine baęlıdır.
- Modern işletim sistemlerinde standart swapping kullanılmaz.
  - Standart swapping, çok fazla swapping süresi gerektirir ve çok az yürütme süresi sağlar.
- Güncellenmiř swapping sürümleri UNIX, Linux ve Windows dahil birçok sistemde bulunur.
  - Yaygın bir olan bir varyasyonda, swapping normalde devre dıřı bırakılır.
  - Ancak boş bellek miktarı bir eşik miktarının altına düřtüęünde başlar.
  - Boş bellek miktarı arttıęında swapping işlemi tekrar durdurulur.

# Context Switch Süresi – Swapping

- Yürütülmek üzere CPU'ya geçecek sonraki process'ler bellekte değilse, bir process'i bellekten çıkarıp (swap out) CPU'ya geçecek process'i belleğe almanız (swap in) gerekir.
- Context switch süresi bu durumda çok yüksek olabilir.
  - Örneğin 4 GB ana belleğe sahip bir bilgisayar sistemimiz ve 1 GB yer kaplayan bir işletim sistemimiz varsa, kullanıcı process'inin maksimum boyutu 3 GB'dir. Ancak, birçok kullanıcı process'i bundan çok daha küçük olabilir - örneğin, 100 MB.
  - 50MB/sn hızındaki bir diskte 3 GB'yi swap out yapmak için 60 sn gerekirken 100 MB'lik bir processi swap out yapmak için 2 sn gerekir.
- Bu yüzden bir kullanıcı process'inin **ne kadar bellek kullanabileceğini değil, tam olarak ne kadar bellek kullandığını** bilmek faydalı olacaktır. Böylece sadece gerçekte kullanılan, swap edilerek swap süresini azaltılabilir.
- Bu yöntemin etkili olması için kullanıcının bellek gereksinimlerindeki herhangi bir değişikliği **request\_memory()** ve **release\_memory()** sistem çağrılarını kullanarak sistemi haberdar etmesi gerekir.

# Yer deęiřtirme (Swapping)

- Swapping, bařka faktörler tarafından sınırlandırılabilir. Bir process'i swap etmek istiyorsak tamamen bořta olduęundan emin olmalıyız.
- Bir process'i swap etmek istedięimizde process bir I/O bekliyor olabilir. I/O, I/O arabellekleri (buffer'leri) için kullanıcının bellek alanına eşzamansız olarak erişiyorsa, process swap edilemez.
- Aygıt meřgul olduęu için I/O işleminin kuyruęa alındıęını varsayalım. P1 process'i çıkarılıp P2 process'i belleęe alınırsa, I/O işlemi řimdi P2 process'ine ait olan belleęi kullanmaya çalışabilir.
- Bu sorunun iki ana çözümü vardır:
  1. Bir process'i hiçbir zaman I/O bekleyen process ile swap etmemek.
  2. I/O işlemlerini yalnızca işletim sistemi arabelleklerinde yürütmek. İşletim sistemi arabellekleri ile process belleęi arasındaki aktarımlar, yalnızca process swap in olmuşsa gerçekleşir.
    - Bu řekilde çift arabellek kullanımı (dubble buffering) ek yük getirir. Kullanıcı process'inin erişebilmesi için verilerin çekirdek belleęinden kullanıcı belleęine kopyalanması gerekir.



# Yer deęiřtirme (Swapping) - Mobilde

- Mobil cihazlar kalıcı saklama birimi olarak hard disk yerine flař bellek kullanır.
- Mobil sistemler swapping iřlemini desteklemez, řünkü:
  - Kısıtlı alan olduęundan mobil iřletim sistemi tasarımcıları swap yapmaktan kařınır.
  - Flař belleklerde yazma sayısı limiti vardır.
  - Mobil cihazlarda, main memory ile flař bellek arasındaki throughput deęeri dūřüktür.

# Yer deęiřtirme (Swapping) - Mobilde

- Bunun yerine, yeteri kadar bellek yoksa belleęi boşaltmak için başka yöntemler kullanılır.
- **Apple iOS** işletim sistemi:
  - Boş bellek belirli bir eřięin altına düřtüęünde, uygulamalardan ayırdıkları belleęi gönüllü olarak bırakmalarını ister.
  - Read-only veri veya kod, sistemden çıkarılır, sonra gerektiğinde flař bellekten tekrar yüklenir.
  - Deęiřebilen veriler (stack) bellekten çıkarılmaz.
  - Yeterli belleęi boşaltamayan uygulamalar işletim sistemi tarafından sonlandırılabilir.
- **Android** işletim sistemi iOS'a benzer stratejiler kullanır, yeterli boş hafıza yoksa bir process'i sonlandırabilir.
  - Ancak, bir process'i sonlandırmadan önce durumunu flash belleęe yazar, böylece hızlı bir şekilde yeniden başlatılabilir.
- Hem iOS hem Android paging'i destekler.

# Konular

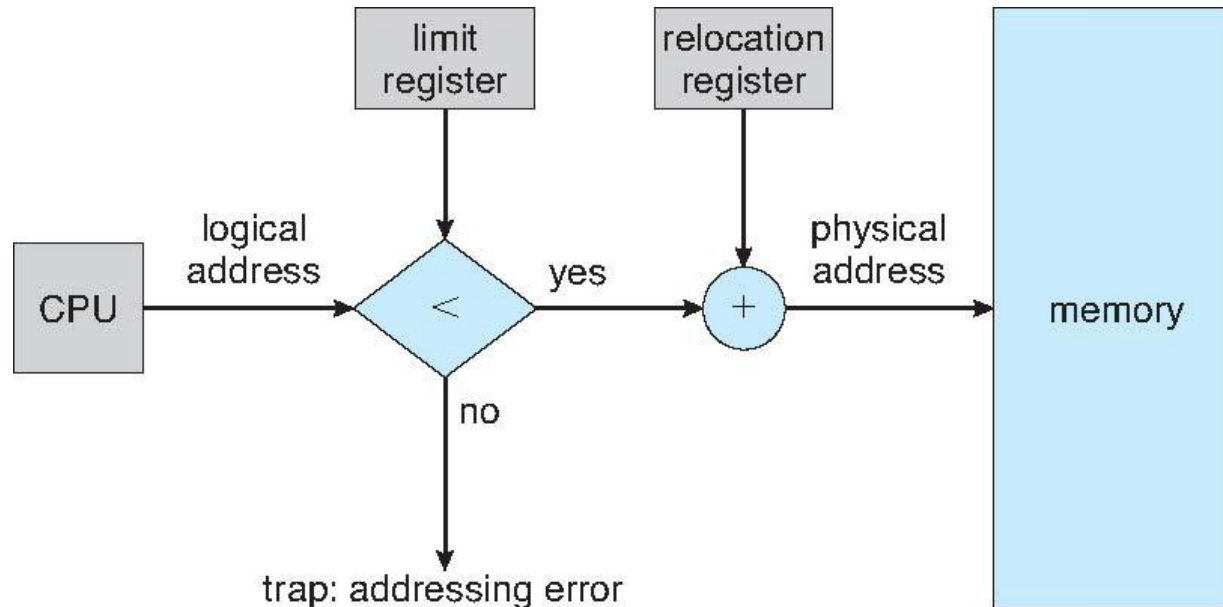
- Giriş
- Swapping
- **Bitişik bellek tahsisi**
- Segmentation
- Paging

# Bitişik bellek tahsisi

- Ana belleğe hem işletim sistemi hem de kullanıcı programlarının yerleştirilmesi gerekir.
- Bu nedenle, ana belleği mümkün olan en verimli şekilde tahsis etmemiz gerekir. Bunun için ilk önerilen yöntemlerden biri **bitişik bellek tahsisi yöntemi**dir (**contiguous memory allocation**).
- Bitişik bellek tahsisi yönteminde bellek iki parçaya ayrılır:
  - İşletim sisteminin yerleştiği kısım
  - Kullanıcı process'lerinin yerleştiği kısım
- İşletim sistemi ile interrupt vektör tablosu genellikle aynı tarafa yerleştirilir. Genellikle de belleğin başlangıcına yerleştirilir.
- Bitişik bellek tahsisi yönteminde, her process, sonraki process'i içeren bölüme bitişik olan, tek bir bellek bölümünde yer alır.

# Bitişik bellek tahsisi - Bellek alanı koruma

- Bir process'in sahip olmadığı bellek alanına erişimini engellemek gerekir.
- Sistemde **relocation register** ve **limit register** ile her process'e kendisine ait bellek alanı ayrılabilir.
- **Relocation register** fiziksel adresin başlangıç değerini, **limit register** mantıksal adresin aralığını tutar.
- MMU, relocation register'ına değer ekleyerek mantıksal adresi dinamik olarak eşleştirir.



# Bitişik bellek tahsisi

- Bir process'e bellek alanı tahsisi, işletim sistemine göre farklı şekillerde yapılabilir.
- En basit yöntemlerden biri çoklu bölmelendirme (**multiple-partition**) metodudur, bellek çok sayıda sabit boyutta küçük bölmelere (**fixed-partitions**) ayrılır.
  - Multiprogramming sistemlerde eşzamanlı çalışan program sayısı bölme sayısına bağlıdır.
  - Bu yapıyı bazı IBM işletim sistemleri (OS/360) kullanmıştır, günümüzde kullanılmamaktadır.
- Diğer bir yöntem olan değişken bölmelendirmede (**variable-partition**) işletim sistemi, belleğin boş ve dolu olan parçalarını bir tabloda tutar.
  - Bölmelendirmelerin boyutları process'lerin ihtiyaçlarına göre yapıldığından daha verimlidir.
  - Boşluk (**hole**) adı verilen kullanılabilir bellek blokları çeşitli boyutlarda hafızaya dağılmıştır.
  - Bir process geldiğinde, sistem process'in ihtiyaç duyduğu bellek kadar büyük bir **hole** arar.
  - Bulunan hole çok büyükse iki parçaya bölünür. Gelen process'e bir parça ayrılır; diğeri hole kümesine geri döndürülür.

# Bitişik bellek tahsisi

- Bir process sona erdiğinde, bellek bloğunu serbest bırakır ve bu daha sonra hole kümesine geri yerleştirilir.
- Yeni hole diğer hole'lere bitişikse, bu bitişik hole'ler birleştirilerek daha büyük bir hole oluşturulur.
- Bu noktada sistemin, hafızayı bekleyen process'ler olup olmadığını ve bu yeni serbest bırakılan ve yeniden birleştirilen belleğin bu bekleyen proces'lerin herhangi birinin taleplerini karşılayıp karşılayamayacağını kontrol etmesi gerekebilir.

# Bitişik bellek tahsisi

- Değişken bölmelendirme (**variable-partition**) yönteminde, bir serbest hole listesinden bir  $n$  boyutunda bellek alanı talebinin nasıl karşılanacağı genel dinamik depolama tahsisi probleminin (**dynamic storage allocation, DSA**) özel bir örneğidir. Bu sorunun birçok çözümü var:
  - **First fit**: Yeterli boyuttaki **ilk** boş alan atanır ve listede kalan kısım aranmaz.
  - **Best fit**: Yeterli boyutta alanların **en küçüğü** seçilir. Liste sıralı değilse tümü aranır.
  - **Worst fit**: Yeterli boyuttaki alanların **en büyüğü** seçilir. Liste sıralı değilse tümü aranır.
- Yapılan simülasyon çalışmaları, hem first fit hem de best fit'in zaman ve depolama kullanımını azaltma açısından worst fit'ten daha iyi olduğunu göstermiştir.



# Bitişik bellek tahsisi - Fragmentation

- Process'ler yüklendikçe ve bellekten kaldırıldıkça, boş bellek alanı küçük parçalara (**fragmentation**) bölünür.
- Dış parçalanma (**External Fragmentation**) – bir isteği karşılamak için yeterli toplam bellek alanı olduğunda, ancak mevcut alanlar bitişik olmadığında oluşur: mevcut alanlar çok sayıda küçük parçalara bölünmüştür.
- Yapılan istatistiksel analize göre; first fit ile  $N$  tane kullanılmış blok için  $N/2$  tane boş blok oluşur. Bu durumda belleğin  $1/3$  kısmı kullanılamaz.
- Dış parçalanma sorununa bir çözüm sıkıştırma (**compaction**). Amaç, tüm boş belleği büyük bir blokta bir araya getirmek için bellek içeriğini yer değiştirmektir.
- Sıkıştırma, yalnızca yer değiştirme dinamikse mümkündür ve yürütme zamanında yapılır.

# Bitişik bellek tahsisi - Fragmentation

- En basit sıkıştırma algoritması, tüm process'leri belleğin bir ucuna taşıyıp tüm boşlukları diğer uca taşıyarak kullanılabilir belleklerin büyük bir blok olarak birleştirilmesidir. Ancak bu yöntem pahalı olabilir.
- Dinamik olarak sıkıştırma yapmak I/O işlemi yapan process'lerde bir sorundur, bir çözüm olarak yalnızca işletim sistemi arabelleğinde (buffer'ında) I/O yapılabilir.
- Dış parçalanma sorununa bir başka çözüm, process'lerin mantıksal adres alanının bitişik olmamasına izin vermektir. Böylece bir belleğin mevcut olduğu her yerde bir process'e tahsis edilmesine izin verilebilir.
  - Bu, bilgisayar sistemleri için en yaygın bellek yönetimi tekniği olan **paging'de (sayfalama)** kullanılan stratejidir.

# Bitişik bellek tahsisi - Fragmentation

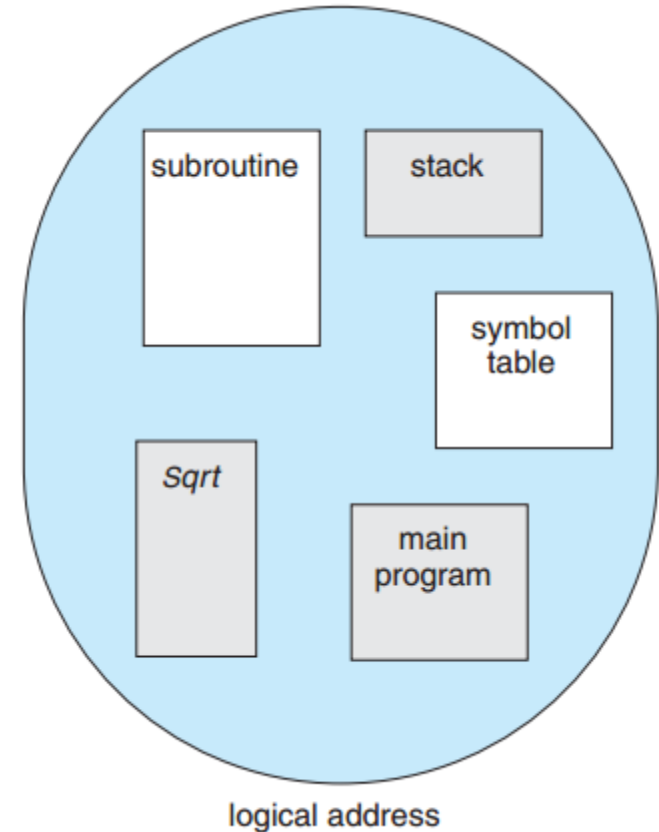
- **İç parçalanma** ([Internal Fragmentation](#)) – ayrılan bellek, istenen bellekten biraz daha büyük olabilir; bu boyut farkı, bir bölümün dahili hafızasıdır, ancak kullanılmamaktadır. Bu duruma **İç parçalanma** denir.
  - Bu sorunu önlemek için fiziksel bellek sabit boyutlu bloklara ayrılabilir ve bellek blok boyutuna göre birimler halinde tahsis edilebilir.
  - Bu yaklaşımla, bir process'e ayrılan bellek istenen bellekten biraz daha büyük olabilir.

# Konular

- Giriş
- Swapping
- Bitişik bellek tahsisi
- **Segmentation**
- Paging

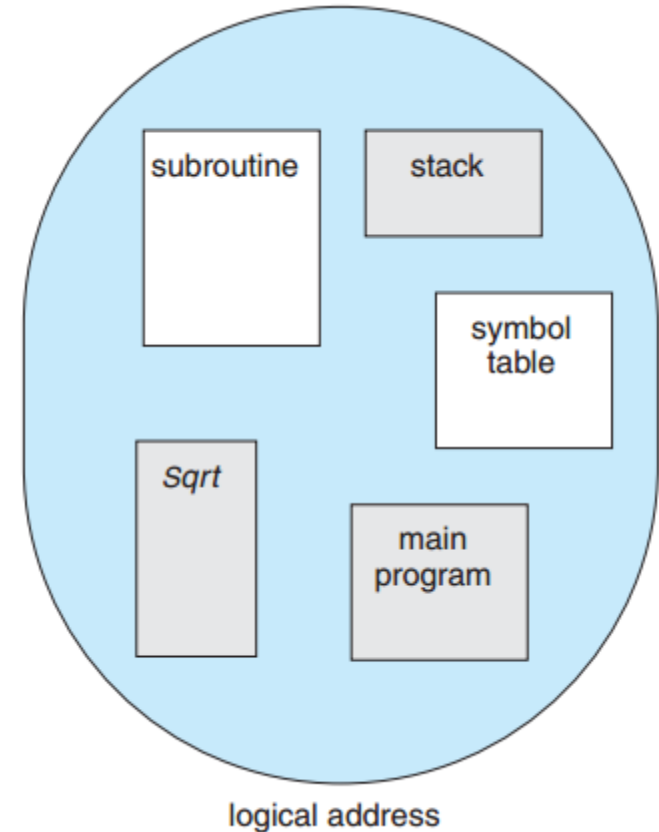
# Bölütleme (Segmentation)

- Kullanıcının bellek olarak gördüğü, fiziksel bellek ile aynı değildir. Bu, programcının bellek olarak gördüğü şey için de geçerlidir.
- Programcılar belleği komutlar ve verilerden oluşan doğrusal bir bayt dizisi olarak düşünmezler.
- Programcılar şekilde görüldüğü gibi belleği, segmentler arasında gerekli bir sıralama olmaksızın, değişken boyutlu segmentlerin bir koleksiyonu olarak görmeyi tercih ederler.



# Bölütleme (Segmentation)

- Bir programcı bir program yazarken programı bir dizi yöntem, prosedür veya fonksiyon içeren bir ana program olarak düşünür.
- Program ayrıca çeşitli veri yapılarını da içerebilir: nesneler, diziler, stack, değişkenler vb. Bu modüllerin veya veri elemanlarının her biri programda **adıyla anılır**.



# Bölütleme (Segmentation)

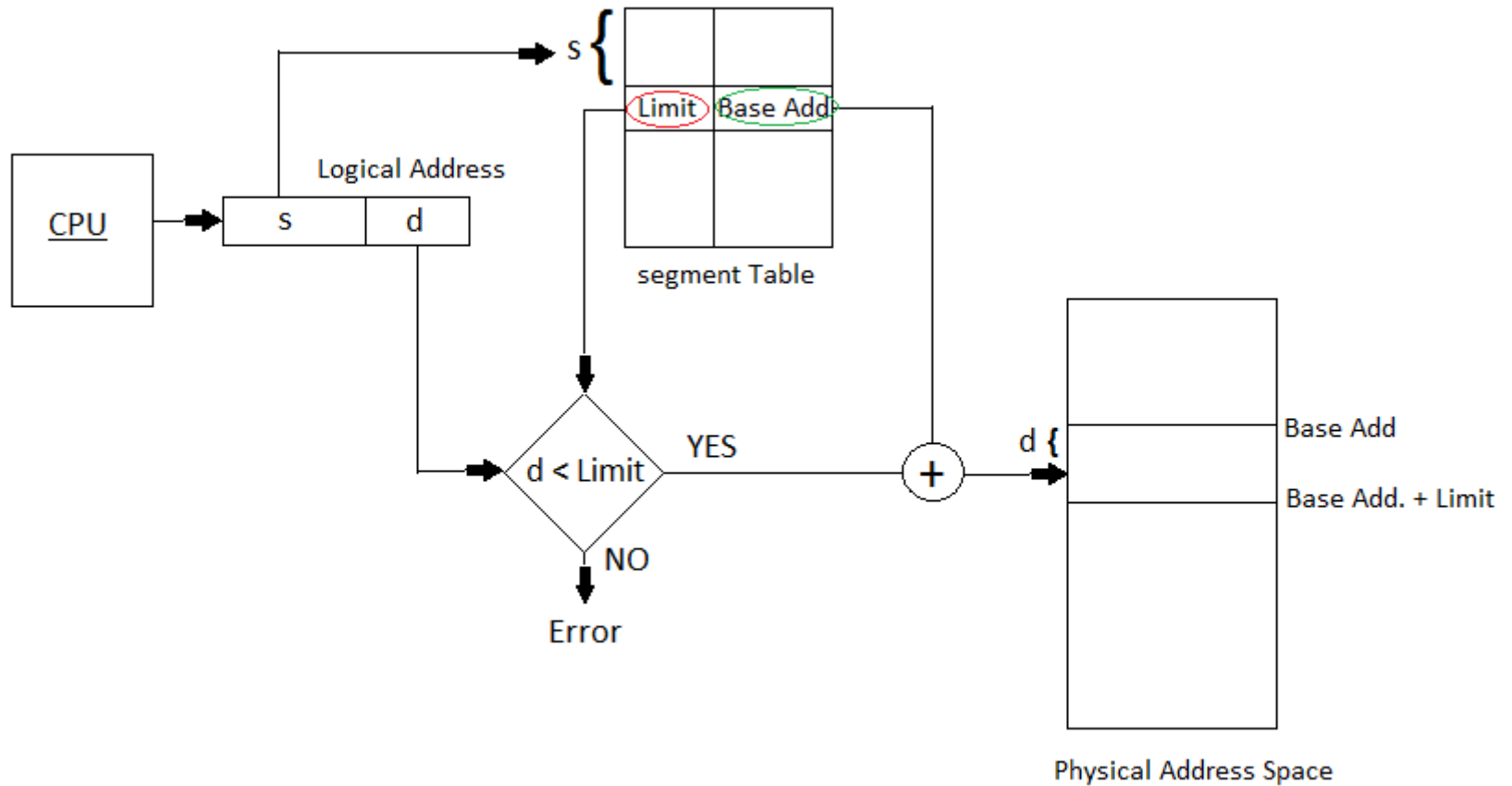
- Fiziksel özellikleri açısından bellekle ilgilenmek hem işletim sistemi hem de programcı için elverişsizdir.
- Bölütleme (**Segmentation**) ile programcının algıladığı bellek, gerçek fiziksel bellekle eşleştirilebilir. **Segmentation** ile programcı daha doğal bir programlama ortamına sahip olurken; işletim sistemi, belleği yönetmek için daha fazla özgürlüğe sahip olur.

# Bölütleme (Segmentation)

- Mantıksal adres alanı, segmentlerin bir koleksiyonudur. Mantıksal bir adres iki kısımdan oluşur: **<segment numarası (s), offset(d)>**
- Derleme sırasında derleyici, segment atamalarını gerçekleştirir.
- Programcı programdaki nesnelere iki boyutlu bir adresle atıfta bulunsa da gerçek fiziksel bellek tek boyutlu bir bayt dizisidir. Bu nedenle, iki boyutlu kullanıcı tanımlı adreslerin tek boyutlu fiziksel adreslerle eşleştirilmesi gerekir.
- Bu eşleştirme, bir segment tablosu tarafından gerçekleştirilir. Segment tablosundaki her girişin bir **segment tabanı (base)** ve bir **segment limiti** vardır.
- **Segment base**, segmentin bellekte bulunduğu başlangıç fiziksel adresini içerir ve **segment limit**, segmentin uzunluğunu belirtir.
- **Segment-table base register (STBR)** segment tablosunun bellekteki konumunu gösterir.
- **Segment-table length register (STLR)** program tarafından kullanılan segment sayısını gösterir.



# Bölütleme – Adresleme Donanımı



# Bölütleme – Adresleme Donanımı

- Şekilde bilgileri verilen segmentlerin fiziksel bellekte başlangıç bitiş noktalarını yazınız.

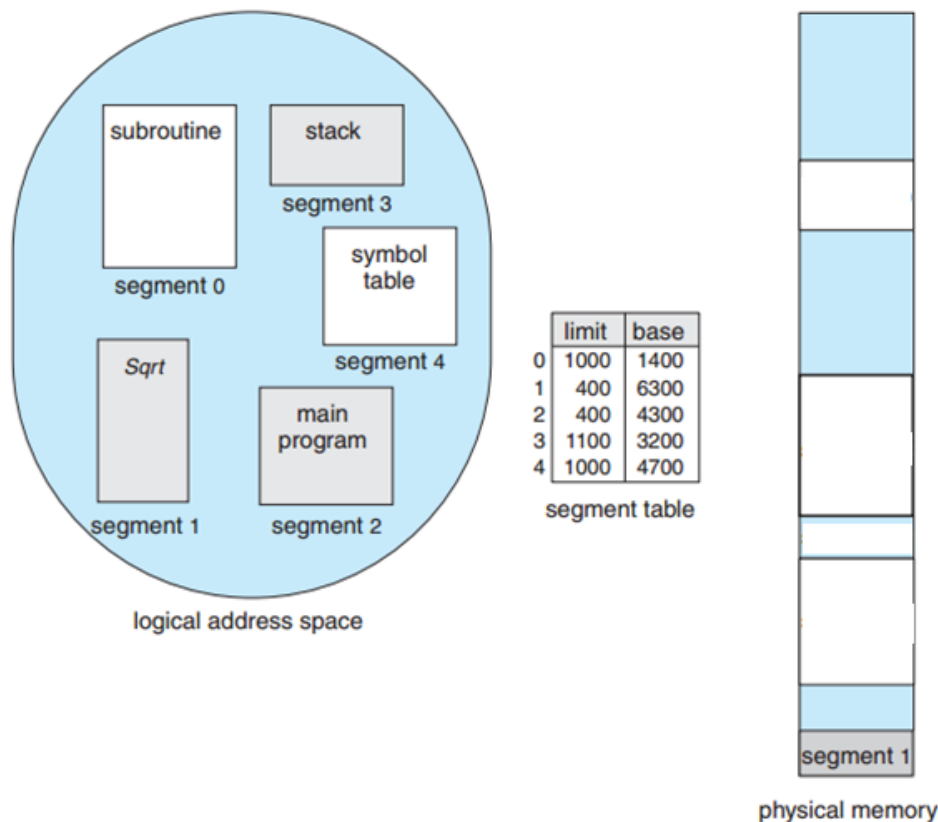


Figure 8.9 Example of segmentation.

# Bölütleme – Adresleme Donanımı

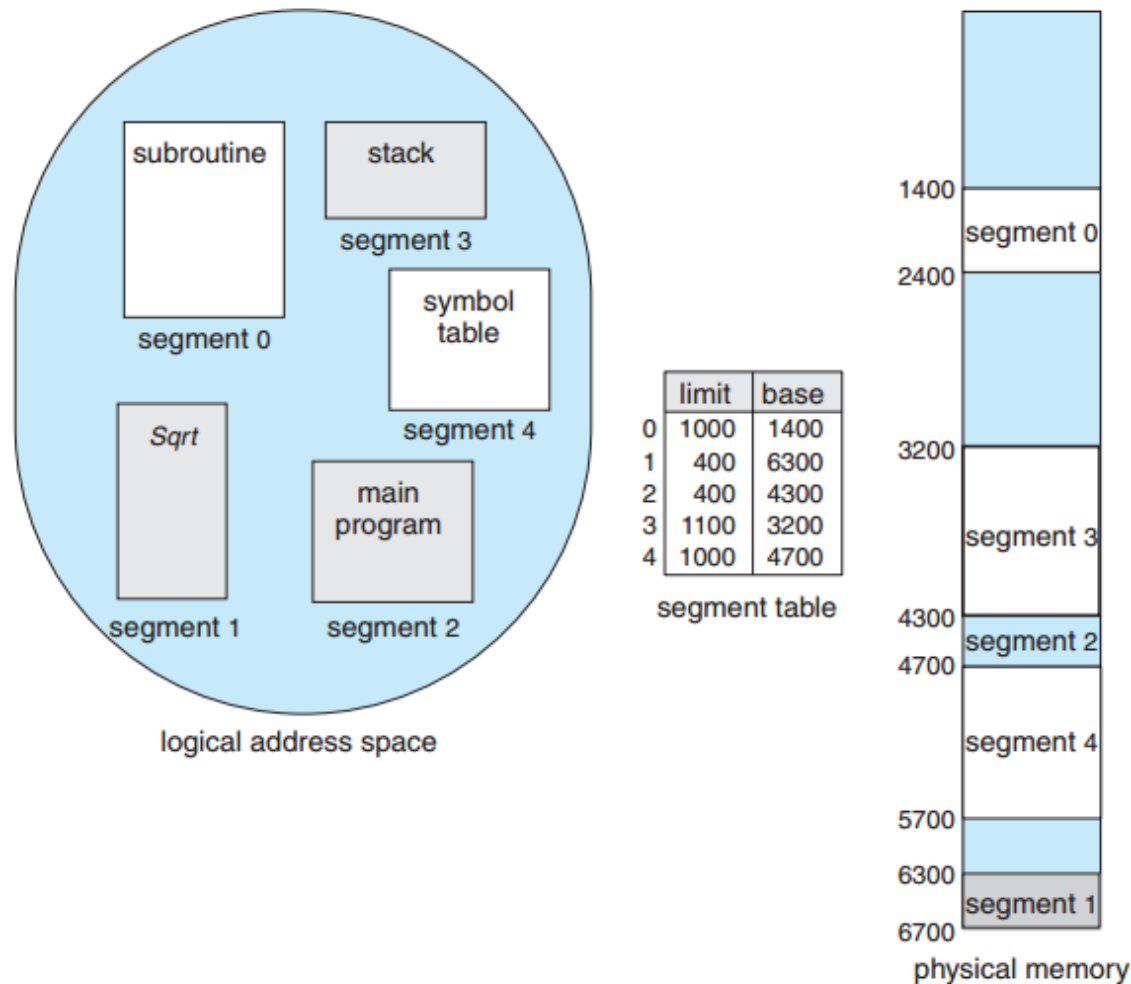


Figure 8.9 Example of segmentation.

# Bölütleme (Segmentation) Mimarisi

- Segmentation'da koruma
  - Segment tablosundaki her giriş bir doğrulama biti ile eşleştirilir, bu bit 0 ise geçersiz segment anlamına gelir.
  - Ayrıca her segment okuma/yazma/ yürütme öncelikleri ile de eşleştirilir.
- Segmentlerle ilişkili koruma bitleri; segment düzeyinde kod paylaşımı yapıldığında gerçekleşir.
- Segmentlerin uzunlukları değiştiğinden bellek tahsisi, dinamik bir depolama tahsisi (dynamic storage-allocation) sorunudur.

# Konular

- Giriş
- Swapping
- Bitişik bellek tahsisi
- Segmentation
- **Paging**

# Sayfalama (Paging)

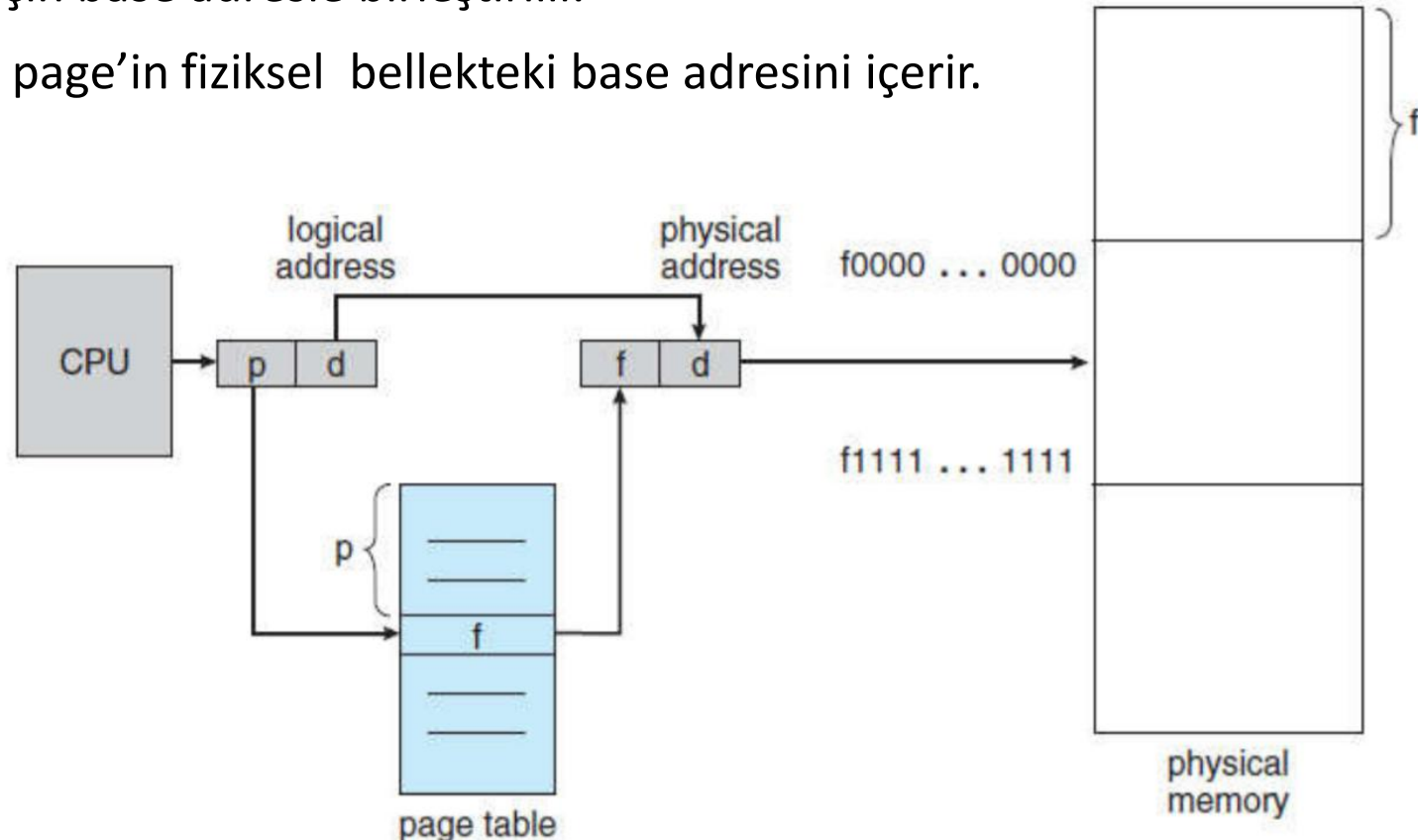
- Segmentation ile bir process'e atanan fiziksel adres alanının bitişik olmamasına izin verilir.
- Sayfalama da (**Paging**) bir başka bellek yönetimi şemasıdır, segmentation'da olduğu gibi process'lere bitişik olmayan bellek adresleri tahsisi avantajı sağlanır.
- Bununla birlikte, **paging** harici parçalanmayı (external fragmentation) ve sıkıştırma (compaction) ihtiyacını ortadan kaldırırken, segmentation bunu yapamaz.
- Aynı zamanda, çeşitli boyutlardaki bellek yığınlarını diske yerleştirme sorununu da çözer. Paging'den önceki şemalarının çoğunda bu sorun söz konusuydu.
  - Sorun, ana bellekteki kodların veya verilerin swap out edilmesi gerektiğinde, bunun için diskte yer bulunmaya çalışıldığında ortaya çıkar.
  - Ana bellekle bağlantılı olarak tartışılan parçalanma sorunları disk için de söz konusudur, ancak erişim çok daha yavaştır, bu nedenle sıkıştırma imkansızdır.

# Sayfalama (Paging)

- Paging yönteminde;
  - Fiziksel bellek, büyüklüğü 2'nin kuvveti olan ve 512 bayt ile 16Mb arasında değişen **frame** adı verilen küçük bloklara bölünür.
  - Mantıksal bellek ise aynı boyutta **page** adı verilen bloklara bölünür.
- Bir process yürütüldüğünde, process'in page'leri diskten herhangi kullanılabilir bir bellek frame'ine yüklenir.
- Disk, bellek frame'leri veya birden çok frame'in kümeleriyle aynı boyutta olan sabit boyutlu bloklara bölünmüştür.
- N tane page'e sahip bir programı çalıştırmak için, N tane boş frame bulmanız ve programı yüklemeniz gerekir.
- Mantıksal adresi fiziksel adreslere çevirmek için bir **page table** oluşturulur.
- İç parçalanma (internal fragmentation) sorunu hala söz konusudur.

# Sayfalama (Paging)

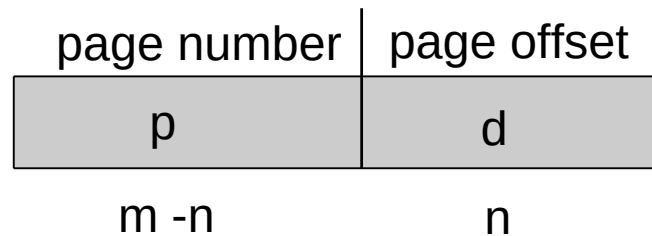
- CPU tarafından oluşturulan her adres iki parçaya ayrılır: **page number(p)** ve **page offset(d)**.
  - **page number(p)** - page table içerisindeki indeks değeri için kullanılır.
  - **page offset(d)** - bellek birimine gönderilen fiziksel bellek adresini tanımlamak için base adresle birleştirilir.
- Page table, her page'ın fiziksel bellekteki base adresini içerir.





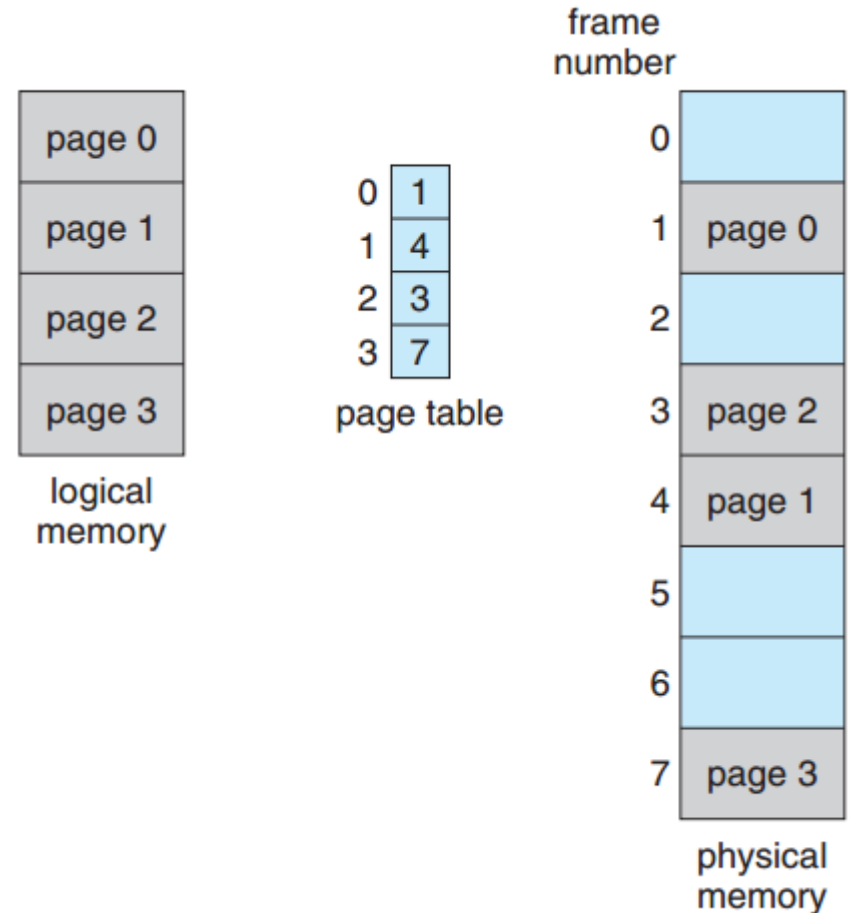
# Sayfalama (Paging)

- Mantıksal adres alanının boyutu  $2^m$  ise ve bir page boyutu  $2^n$  bayt ise, bu durumda
  - mantıksal bir adresin yüksek öncelikli  $m-n$  bitleri  $\rightarrow$  page numarasını
  - $n$  düşük öncelikli biti  $\rightarrow$  page offsetini belirler.
- Dolayısıyla mantıksal adres aşağıdaki gibidir:



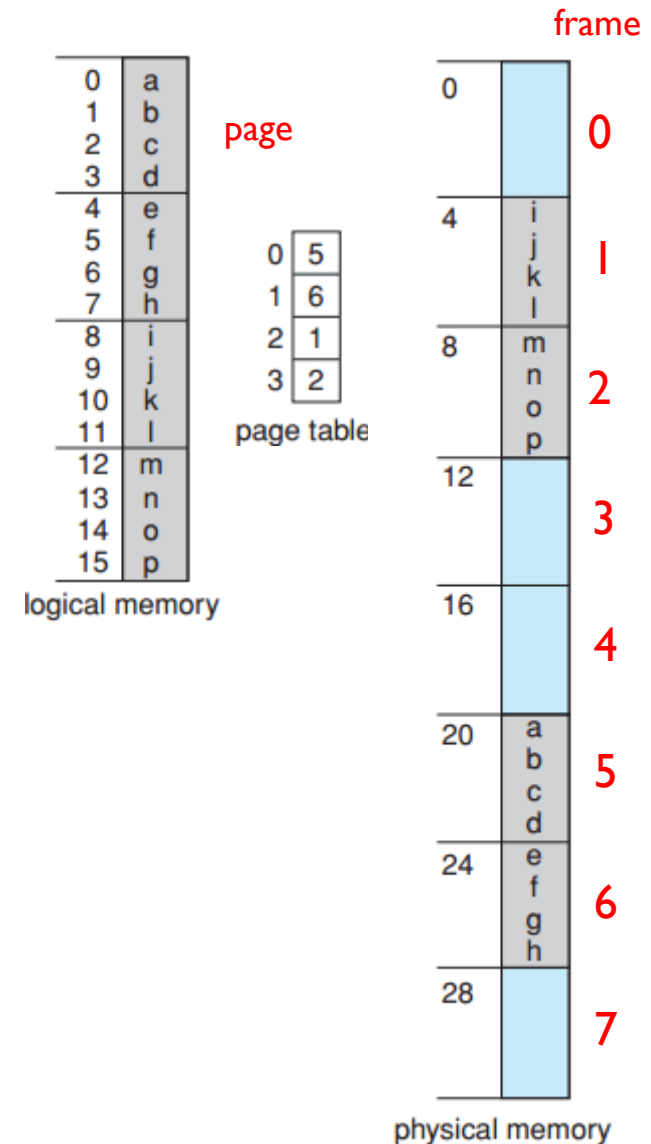
# Sayfalama (Paging)

- Paging model şekilde gösterilmiştir. Page boyutu (frame boyutu gibi) donanım tarafından belirlenir.
- Bir page'in boyutu, bilgisayar mimarisine bağlı olarak page başına 512 bayt ile 1 GB arasında değişen 2'nin kuvveti şeklindedir.



# Sayfalama (Paging)

- Şekildeki örnekte programcının gördüğü belleğin fiziksel bellekle nasıl eşleştirildiği gösterilmiştir.
- **Şekildeki mantıksal adres için m ve n bitleri kaçtır?**
- **3 nolu mantıksal adresin fiziksel adresi kaçtır?**



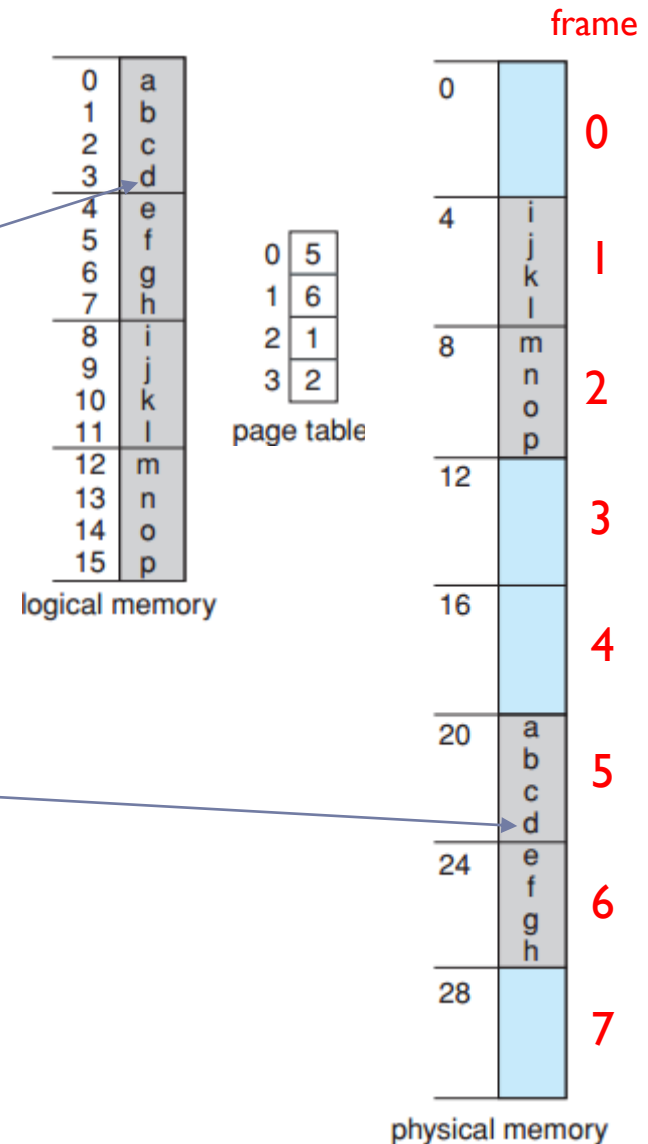
# Sayfalama (Paging)

- Şekildeki örnekte programcının gördüğü belleğin fiziksel bellekle nasıl eşleştirildiği gösterilmiştir.
- Mantıksal adres için  $n=2$  ve  $m=4$  bittir. Page tablosunda  $m-n=2$  bit ile 4 farklı frame tutulabilir,  $n=2$  bit ile 4 farklı offset tutulabilir.
- Mantıksal adres 3 için 0011

Page numarası=0

Offset=3

- Page tablosunda 0. indeks 5. frame'i gösterir, 5. frame 20. adres'ten başlar, 3 offset değeri de eklenince 23. fiziksel adres ile eşleşir.



# Sayfalama (Paging)

- Paging ile iç parçalanma (internal fragmentation) oluşabilir.
- **Aşağıdaki örnekte prosese tahsis edilecek bellek alanında kaç bayt iç parçalanma olur?**
  - Page size = 2048 bayt  
process size = 72766 bayt olduğunda

# Sayfalama (Paging)

- Page size = 2048 bayt  
process size = 72766 bayt olduğunda  
Gerekli alan = 35 page + 1086 bayt olur  
1086 bayt 36. page'e yerleştirilir.
- Kullanılmayan alan  $2048 - 1086 = 962$  bayt olur, 36. page'de 962 bayt boş alan kalır.

# Sayfalama (Paging)

- En kötü durumda bir process için kaç baytlık iç parçalanma olur?
  - Page size = 2048 bayt

# Sayfalama (Paging)

- En kötü durum için bir process'in  $[n \text{ page} + 1 \text{ bayt}]$  ihtiyacı olacaktır. Bu durumda  $N + 1$  frame tahsis edilir ve  $N+1$  nolu frame 'in neredeyse tümü iç parçalanmasıyla sonuçlanır.
  - Yukarıdaki örnek için en kötü durum 2049 baytlık process'te olur.
  - 2048 bayt bir page'e yerleştirilir, kalan 1 bayt ikinci bir page'e yerleştirilir.
  - Boş kalan en kötü durum  $2048 - 1 = 2047$  bayt olur.

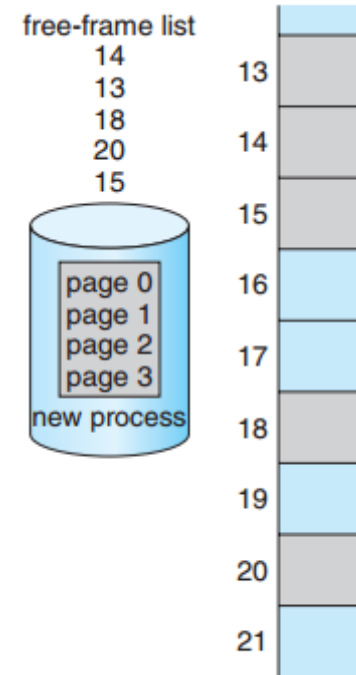


# Sayfalama (Paging)

- Process boyutu page boyutundan bağımsızsa, iç parçalanmanın process başına ortalama  $n/2$  page olmasını beklenir.
  - Bu durumda küçük page boyutları arzu edilir.
  - Bununla birlikte page tablosuna her kayıt girişinde ek yük söz konusu olacaktır, bu ek yük, page'lerin boyutu arttıkça azalır.
- Process'lerin veri kümeleri ve ana bellek büyüdükçe genellikle page boyutları zamanla büyümüştür.
  - Hatta bazı CPU'lar ve çekirdekler birden çok page boyutunu destekler.
    - Örneğin Solaris, page'lerin depoladığı verilere bağlı olarak 8 KB ve 4 MB page boyutlarını kullanır.

# Sayfalama (Paging)

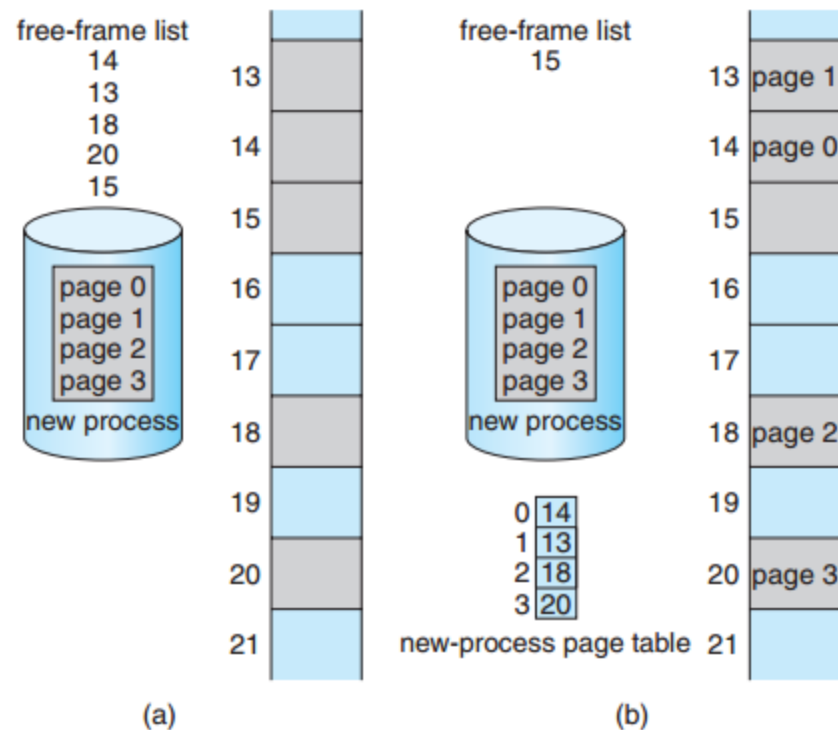
- Bir process sisteme çalışmak için geldiğinde, gerekli page sayısı belirlenir.
- Process'in her page'i bir frame'e ihtiyaç duyar.
  - Toplam  $n$  page varsa, en az  $n$  tane frame boş olmalıdır.
  - Her page bir frame'e yerleştirilir ve frame numarası page table'a kaydedilir.
- Şekilde yeni oluşturulan bir new prosesin ihtiyaç duyduğu page sayısı ve belleğin anlık görünümü verilmiştir.
- New proses için uygun bir page tablosunu oluşturun ve new proses'in page'lerinin page tablosuna göre bellekte hangi frame'lere yerleştirileceğini gösteriniz?



(a)

# Sayfalama (Paging)

- Bir process sisteme çalışmak için geldiğinde, gerekli page sayısı belirlenir.
- Process'in her page'i bir frame'e ihtiyaç duyar.
  - Toplam  $n$  page varsa, en az  $n$  tane frame boş olmalıdır.
  - Her page bir frame'e yerleştirilir ve frame numarası page table'a kaydedilir.
- Şekilde bir process'e ait page'lerin hafızaya yerleştirilmesi görülmektedir.
  - (a) new process yerleşmeden önceki boş frame'ler
  - (b) process yerleştikten sonraki bellek görünümü
- Programcı, process'in adresini tek ve bitişik olarak görür.
  - Frame eşleştirmesini işletim sistemi yapar.



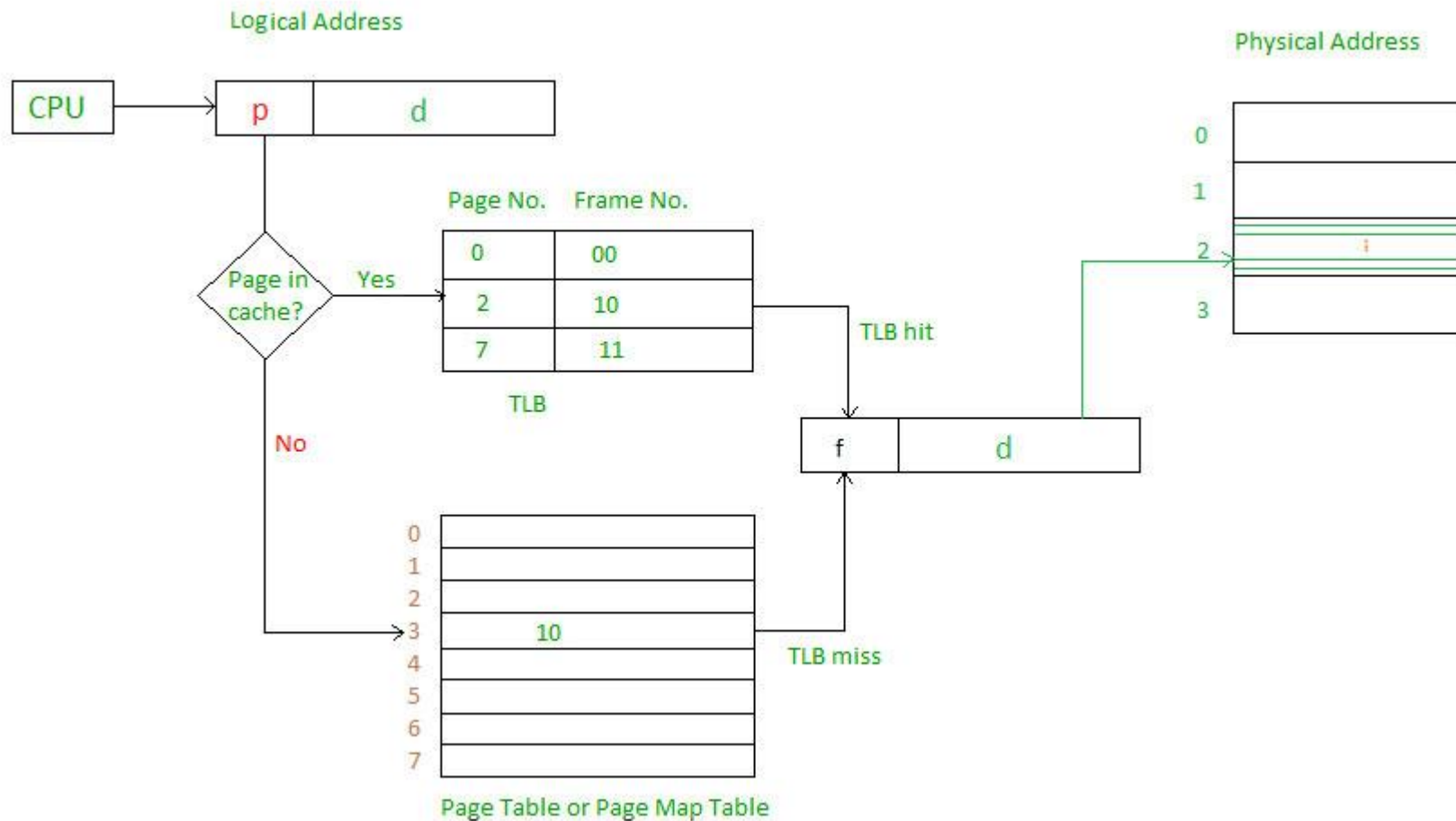
# Sayfalama (Paging) - Page Tablosunun Donanım İle Gerçekleştirimi

- Page tablosu ana bellekte tutulur.
- **Page table base register (PTBR)** page tablosunu gösterir.
- **Page table length register (PTLR)** page tablosunun boyutunu gösterir.
- Bu şemada her veri veya komut erişimi, belleğe iki kez erişimi gerektirir.
  - Biri page tablosu için, diğeri veri veya komut içindir
- Belleğe iki kez erişim sorunu, **associative memory** yada **translation look-aside buffers (TLBs)** adı verilen özel bir fast-lookup donanım önbelleği kullanılarak çözülebilir.
  - TLB; ilişkilendirilebilir, yüksek hızlı bellektir. TLB'deki her giriş iki bölümden oluşur: bir anahtar (page number) ve bir değer (frame).

# Sayfalama – Translation look-aside buffer

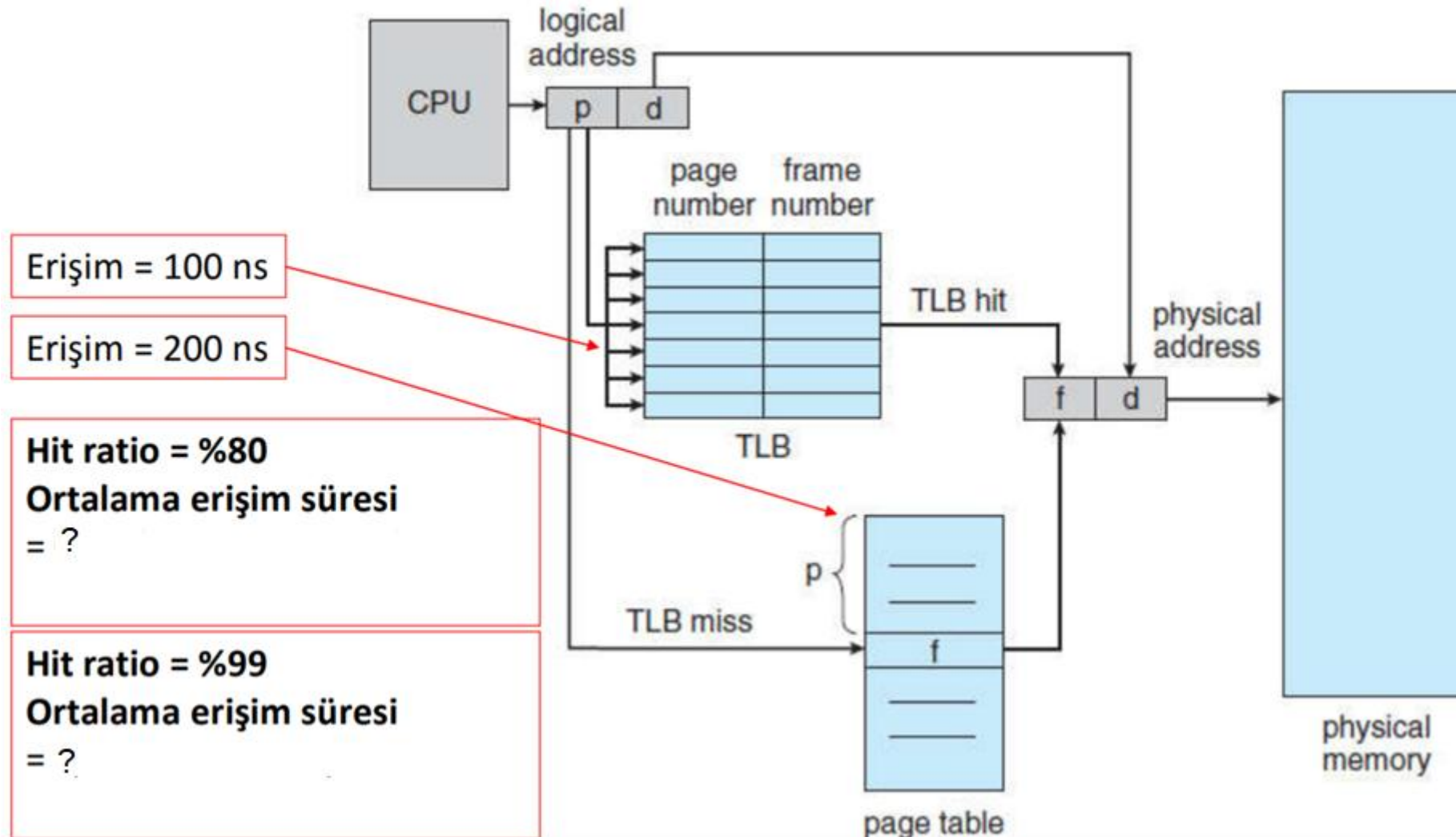
- Bazı TLB'lerde belirli girişlerin **wired down** olmalarına izin verilir, bu da bunların TLB'den kaldırılamayacağı anlamına gelir. Böylece bunlara sürekli hızlı erişilebilmeleri sağlanır.
- Bazı TLB'ler, her TLB girişinde **address-space identifiers (ASIDs)** depolar. Bir
  - ASID, process'lere adres alanı koruması sağlamak için kullanılır, her process'i benzersiz (uniquely) bir şekilde tanımlar.
- TLB ASID'leri desteklemiyorsa, yeni page tablosu her seçildiğinde bir sonraki yürütülecek process'in yanlış adres dönüşümünü kullanmasının önüne geçmek için TLB'nin temizlenmesi gerekir.

# Sayfalama – Translation look-aside buffer



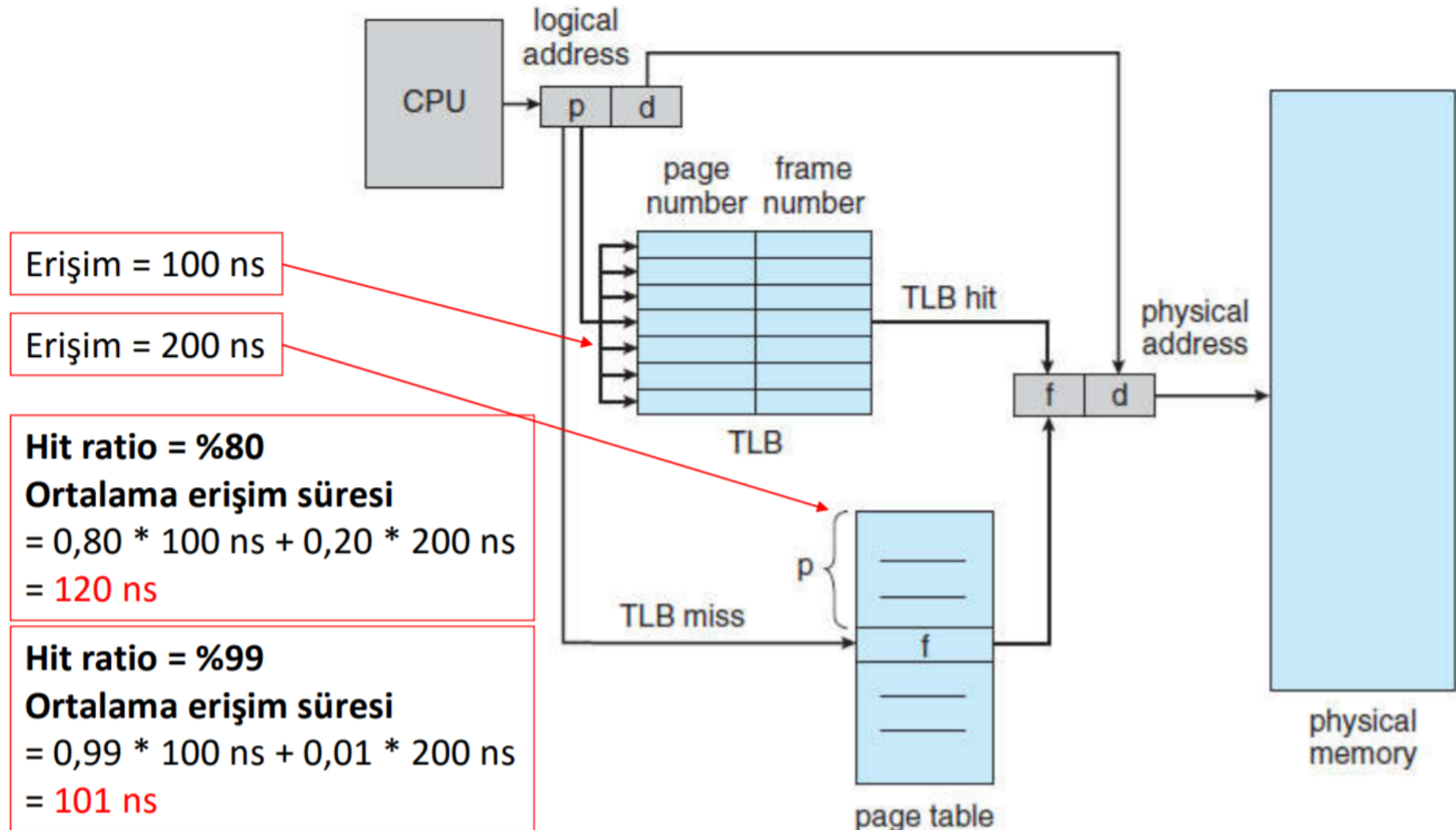
# Sayfalama – Translation look-aside buffer

- Page numarasının TLB'de bulunma yüzdesine isabet oranı (**hit ratio**) denir.



# Sayfalama – Translation look-aside buffer

- Page numarasının TLB'de bulunma yüzdesine isabet oranı (**hit ratio**) denir.



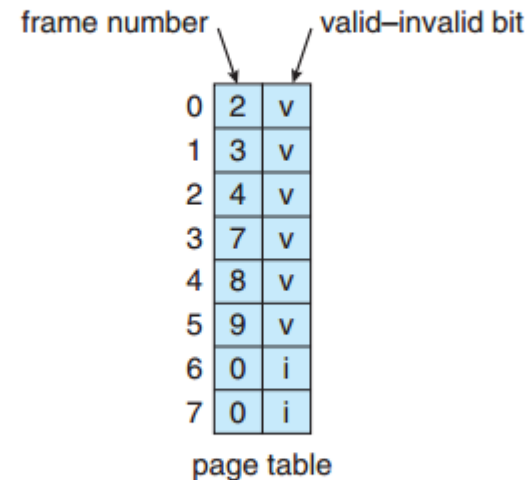
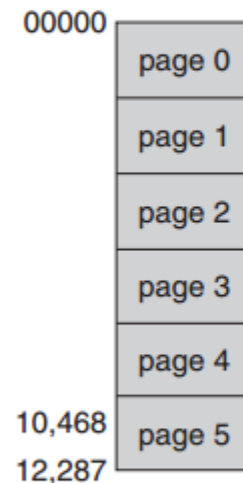


# Sayfalama – Bellek Koruması

- Bellek koruması, her frame salt okunur olduğunu belirtmek için veya okuma-yazma erişimine izin verilip verilmediğini belirtmek için, koruma biti ilişkilendirilerek uygulanır.
  - Ayrıca, page'lerin execute-only olup olmadığını belirtmek için ve daha başka durumları belirtmek için daha fazla bit eklenebilir.
- Page tablosuna her giriş için **Valid-invalid** biti eklenir:
  - "valid", ilişkili page'in process'in mantıksal adres alanında olduğunu ve dolayısıyla geçerli bir page olduğunu belirtir.
  - "invalid", page'in process'in mantıksal adres alanında olmadığını gösterir.
- **Valid-invalid** biti yerine, **Page table length register (PTLR)** da kullanılabilir. Bu register'in değeri adresin process için geçerli aralıkta olup olmadığını doğrulamak için her mantıksal adresle karşılaştırılır.
- Herhangi bir ihlal, çekirdekte bir hatayla sonuçlanır.

# Page Tablosunda Valid (v) İnvaid (i) Biti

- Örneğin, 14 bit adres alanına (0 ila 16383) sahip bir sistemde, yalnızca 0 ila 10468 adreslerini kullanan bir programımız olsun.
- 2 KB'lik bir page boyutu verildiğinde şekilde gösterilen duruma sahip oluruz.
- 0, 1, 2, 3, 4 ve 5. page'lerdeki adresler normal olarak page tablosu aracılığıyla eşleştirilir.
- 6. veya 7. page'lerde bir adrese erişilmeye çalışıldığında invalid biti ile karşılaşılacaktır ve işletim sistemi hata verecektir.

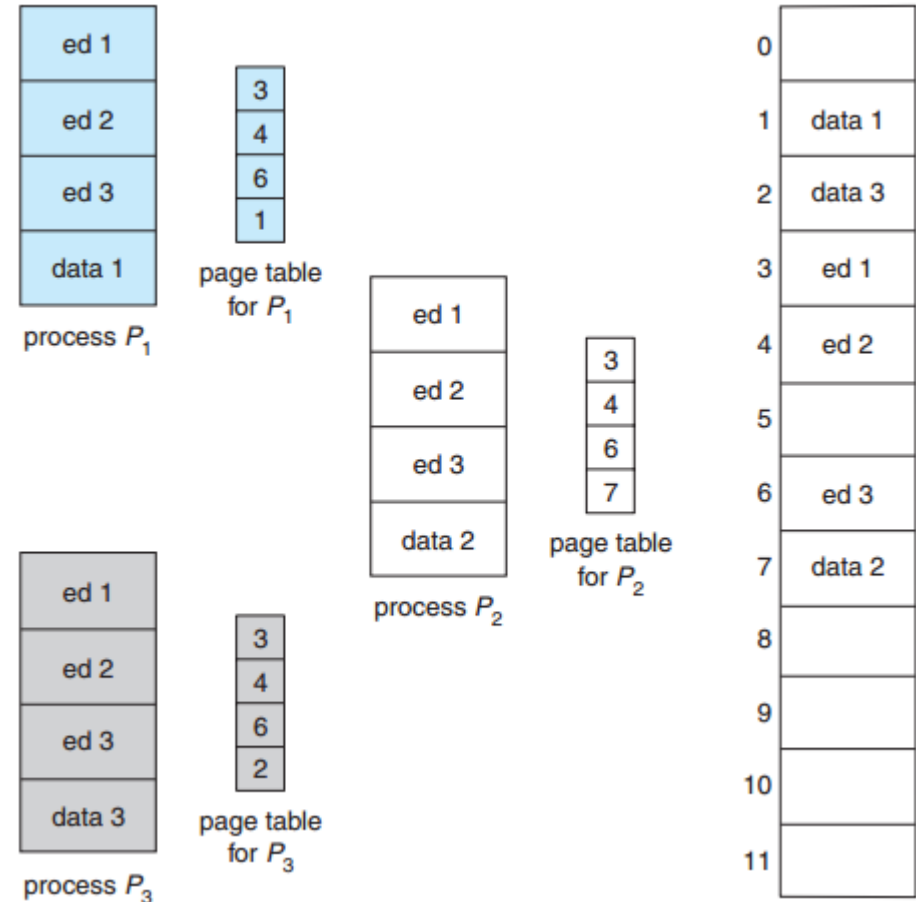


# Paylaşılan Sayfalar

- Paylaşılan kod
  - Paging'in bir avantajı, ortak kodu paylaşma olasılığıdır. Bu, zaman paylaşımli (time-sharing) sistemlerde özellikle önemlidir.
  - Process'ler arasında salt okunur (**reentrant**) kodun bir kopyası paylaşılır (ör. Metin düzenleyiciler, derleyiciler, window sistemleri)
  - Bu, aynı process alanının (space) birden çok iş parçacığıyla paylaşılmasına benzer.
  - Okuma-yazma page'lerinin paylaşımına izin veriliyorsa, process'ler arası iletişim için de kullanışlıdır.

# Paylaşılan Sayfalar

- Her biri bir text editör çalıştıran 40 kullanıcıyı destekleyen bir sistem olsun.
- Text editör, 150 KB kod ve 50 KB veri alanından oluşuyorsa, 40 kullanıcıyı desteklemek için 80MB'a ihtiyacımız var.
- Kod, reentrant kod (veya salt okunur) ise yani yürütülmesi sırasında asla değişmeyen kod ise, şekilde gösterildiği gibi paylaşılabilir.
- Burada, üç page yer kaplayan bir text editörü paylaşan üç process görüyoruz - her page'in boyutu 50 KB.
- Her process'in kendi veri page'i vardır. Böylece iki veya daha fazla process aynı anda aynı kodu çalıştırabilir.

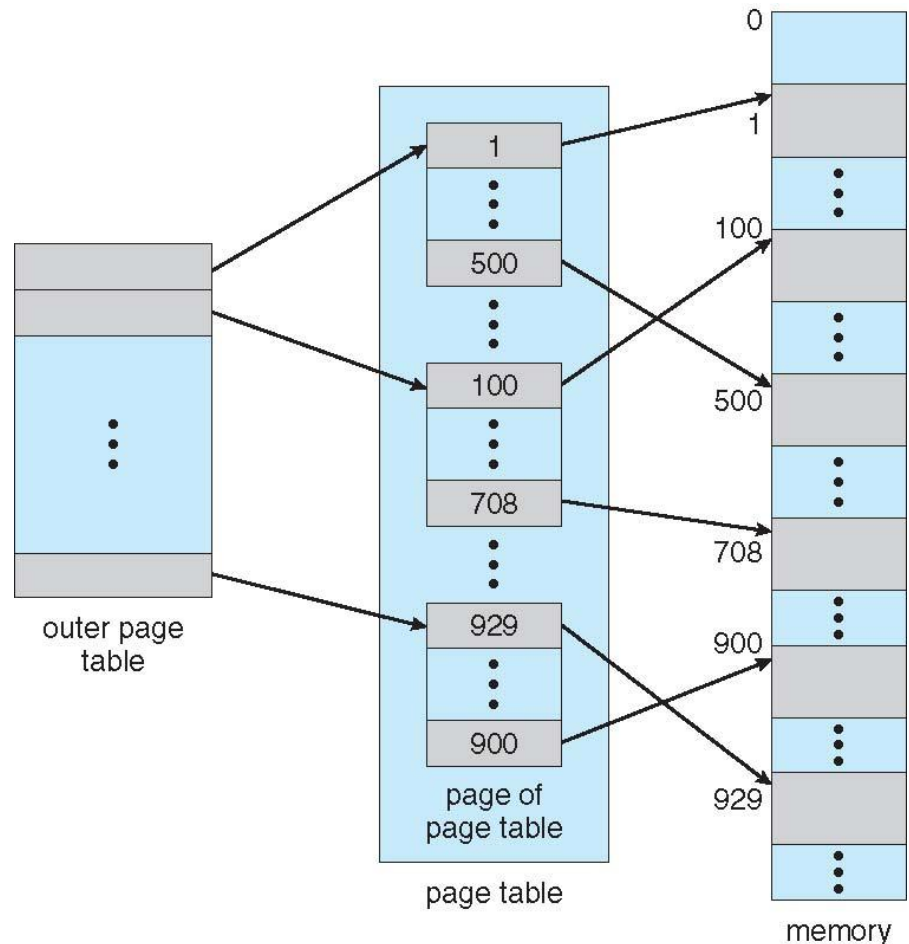


# Page Tablosunun Yapısı

- Paging için bellek yapıları, basit yöntemler kullanıldığında çok büyük olabilir.
  - Örneğin, 32-bit mantıksal adres alanına sahip bir sistem düşünün. Böyle bir sistemde page boyutu 4 KB ( $2^{12}$ ) ise, o zaman bir page tablosu 1 milyona kadar girişten ( $2^{32}/2^{12}$ ) oluşabilir.
  - Her bir girişin 4 bayttan oluştuğu varsayıldığında, her process tek başına page tablosu için 4 MB'a kadar fiziksel adres alanına ihtiyaç duyabilir.
  - Bu, çok pahalıya mal olan bir bellek miktarıdır. Page tablosunun ana bellekte bu boyutlarda bitişik olarak tahsis edilmesi istenmez.
- Bu sorunun çözümü için bazı yöntemler vardır:
  - Hierarchical Paging
  - Hashed Page Tables
  - Inverted Page Tables

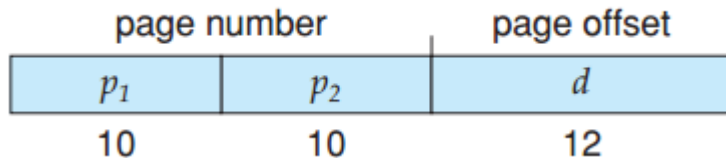
# Hierarchical Paging

- Mantıksal adres alanı birden çok page tablosuna bölünür.
- Bu bölünme birkaç şekilde yapılabilir. Bunun bir yolu:
  - page tablosunun kendisinin de paging edildiği şekilde iki seviyeli bir paging algoritması kullanmaktır.

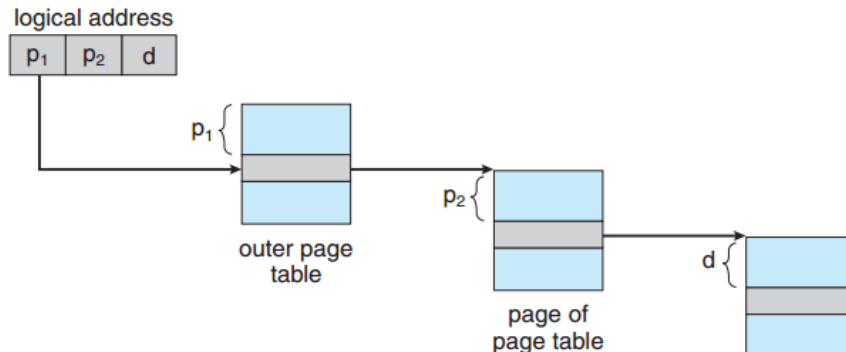


# Two-Level Paging

- Örneğin, 32 bit mantıksal adres alanına ve 4 KB page boyutuna sahip bir sistem düşünün.
- Mantıksal bir adres, 20 bitten oluşan bir page numarasına ve 12 bitten oluşan bir page ofsetine bölünsün.
- Page tablosunu paging ettiğimiz için, page numarası ayrıca 10 bitlik bir page numarasına ve 10 bitlik bir page ofsetine bölünsün.
- Dolayısıyla mantıksal bir adres aşağıdaki gibidir:

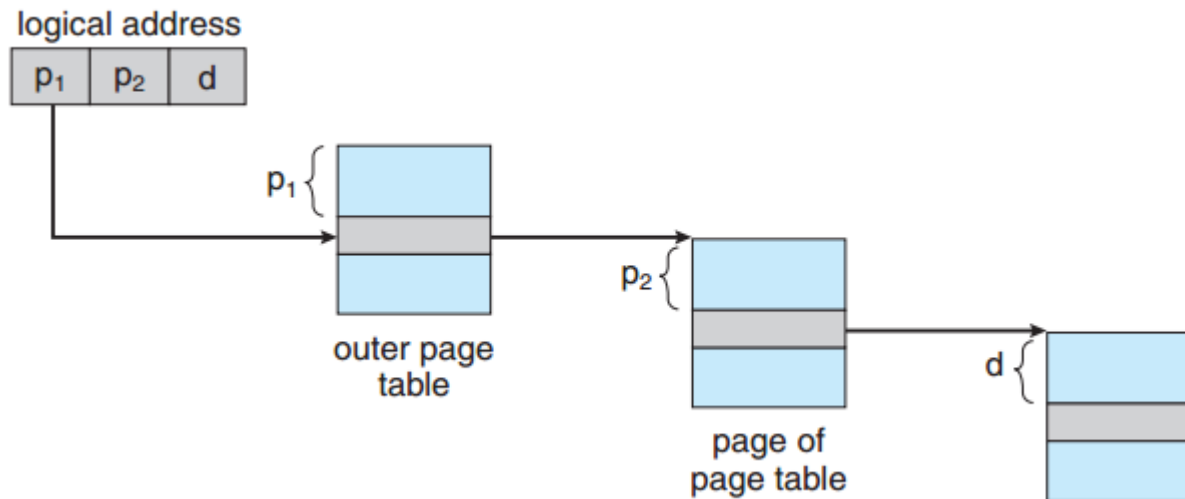


- burada  $p_1$ , dıştaki page tablosunda  $p_2$  ise içteki page tablosunda bir indekstir.



# İki Seviyeli Sayfalama (Two-Level Paging)

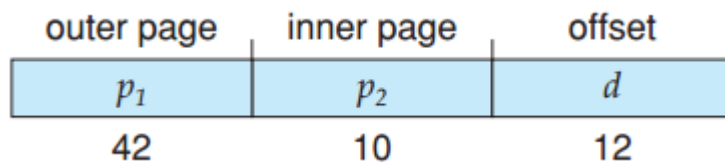
- Bu mimari için adres dönüştürme yöntemi şekilde gösterilmektedir.
- Adres dönüştürme dıştaki page tablosundan içe doğru çalıştığı için, bu şema **forward-mapped page table** olarak da bilinir.





# 64-bit Mantıksal Adres Alanı

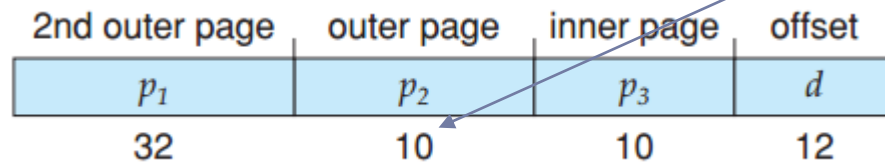
- 64 bit mantıksal adres alanına sahip bir sistem için iki seviyeli paging şeması uygun değildir.
- Bunu açıklamak için page boyutunun 4 KB ( $2^{12}$ ) olduğu bir sistem yine varsayalım. Bu durumda, page tablosu en fazla  $2^{52}$  ( $2^{64}/2^{12}$ ) girişten oluşur.
- İki seviyeli paging kullanırsak, içteki page tabloları uygun page uzunluğunda olabilir,  $2^{10}$  adet 4 baytlık giriş içerebilir. Adresler şuna benzer:



- Dıştaki page tablosu  $2^{42}$  girişten veya  $2^{44}$  bayttan oluşur. Bu kadar büyük bir tablodan kaçınmak için dıştaki page tablosu daha küçük parçalara bölünebilir.

# Üç Seviyeli Sayfalama (Three-Level Paging)

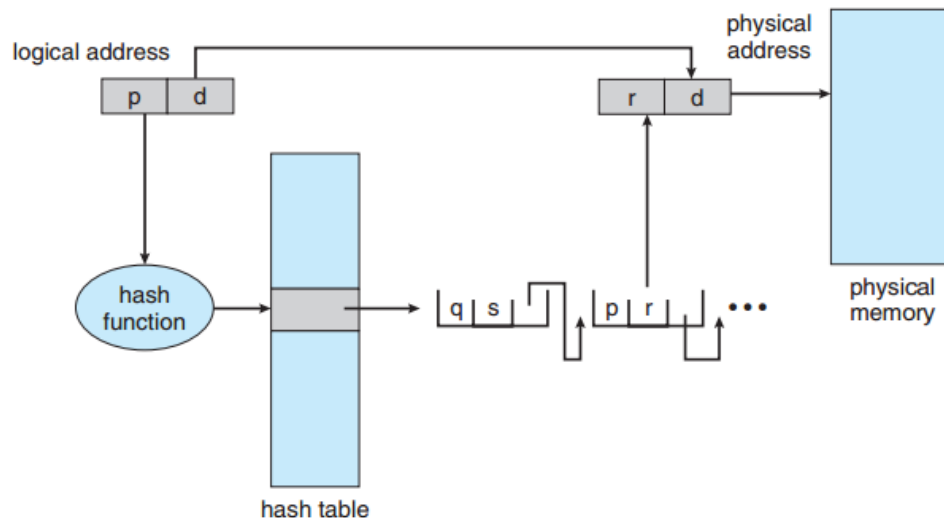
- Dıştaki page tablosu çeşitli şekillerde bölünebilir. Örneğin dıştaki page tablosu üç seviyeli bir paging elde edecek şekilde paging yapılabilir.
- Dıştaki page tablosunun standart boyutlu page'lerden ( $2^{10}$  giriş veya  $2^{12}$  bayt) oluştuğunu varsayalım. Bu durumda, 64 bitlik bir adres alanı hala istenmeyen bir büyüklüktedir:



- En dıştaki page tablosu hala  $2^{34}$  bayt (16 GB) boyutundadır.
- Bir sonraki adım dört seviyeli bir paging şeması olacaktır. Dolayısıyla 64 bit mimariler için hiyerarşik page tablolarının genel olarak neden uygun olmadığı görülmektedir.

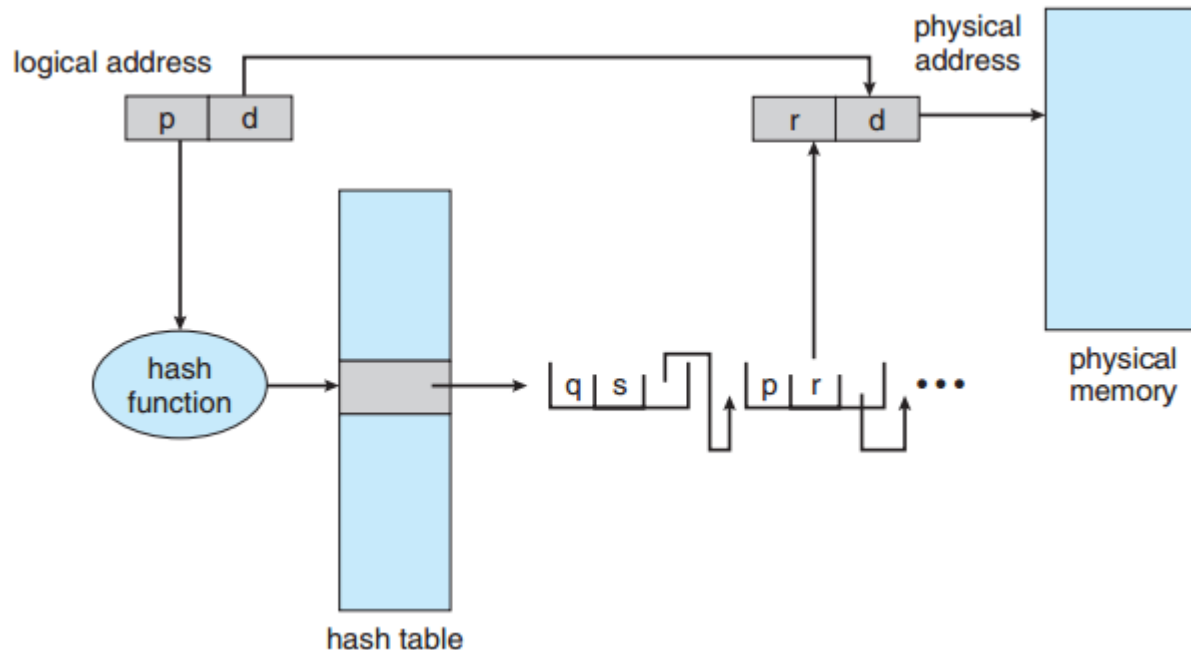
# Hashed Page Tables

- 32 bitten daha büyük adres alanlarının işlenmesine yönelik yaygın bir yaklaşım, sanal page numarasının bir hash değeri olarak tutulduğu **hashed page tablosu** kullanmaktır.
- Hash tablosundaki her giriş bağlı liste ile birbirine bağlı elemanlardan oluşur, her girişin elemanları aynı hash konumundadır.
- Her eleman üç alandan oluşur:
  1. sanal page numarası,
  2. eşlenen page frame'ın değeri
  3. bağlantılı listedeki sonraki elemanı gösteren bir işaretçi (pointer)



# Hashed Page Tables

- Algoritma şu şekilde çalışır: Sanal adresteki sanal page numarası, hash tablosunda hash olarak yer alır.
- Sanal adresin fiziksel adres karşılığı bulunurken sanal adresin sanal page numarası, bağlantılı listedeki ilk elemanın 1. alanı ile karşılaştırılır. Bir eşleşme varsa, istenen fiziksel adresi oluşturmak için karşılık gelen page frame (2. alan) kullanılır.
- Eşleşme yoksa, bağlı listedeki sonraki elemanlar eşleşen bir sanal page numarası için aranır. Bu şema şekilde gösterilmektedir.



# Hashed Page Tables

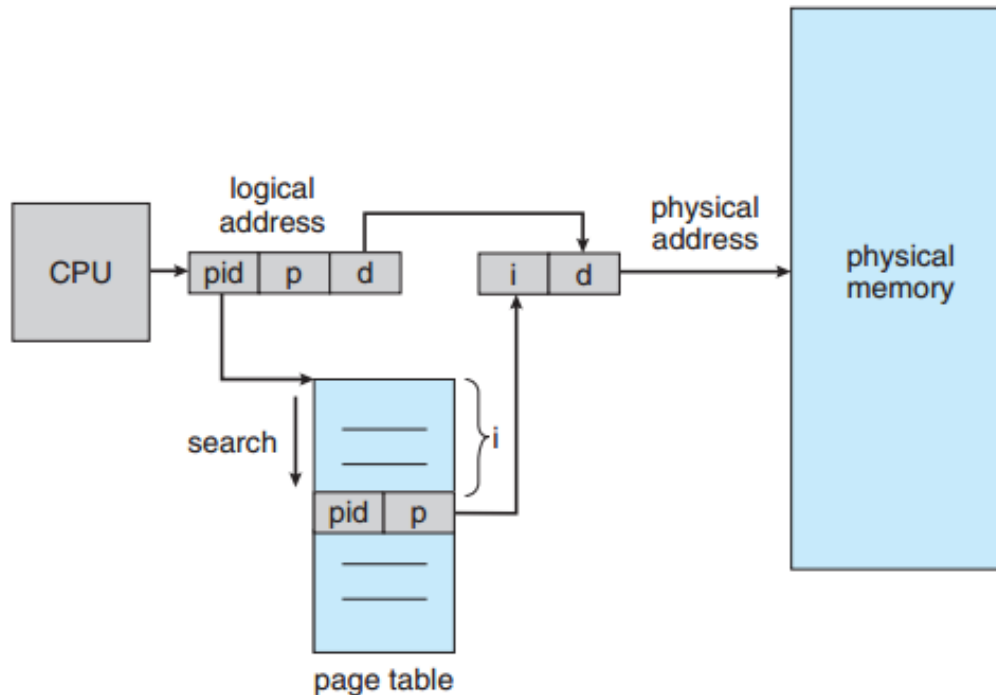
- Bu şemanın 64-bit adres alanları için önerilmiş kullanışlı bir varyasyonu şöyledir:
  - Bu varyasyon hash tablolara benzeyen küme page tabloları (**clustered page tables**) kullanır, ancak her giriş hash tablodaki gibi tek bir page yerine birkaç page'e atıfta bulunur.
  - Bu nedenle, tek bir page tablosu girişi, birden çok fiziksel page frame için eşlemeleri depolayabilir.
  - Küme page tabloları, bellek referanslarının bitişik olmadığı ve adres alanı boyunca dağınık olduğu aralıklı (**sparse**) adres alanları için özellikle kullanışlıdır.

# Inverted Page Tables

- Önceki paging şemalarında genellikle **her process'in ilişkili bir page tablosu vardır**. Page tablosunda, process'in kullandığı her page için bir kayıt vardır.
- Bu yöntemin dezavantajlarından biri, her page tablosunun milyonlarca kayıttan oluşabilmesidir.
- Sadece fiziksel belleğin nasıl kullanıldığını takip etmek için kullanılan bu page tabloların kendisi büyük miktarda fiziksel bellek tüketebilir.
- Bu sorunu çözmek için **inverted page tablosu** kullanılabilir:
  - Inverted page tablosunda belleğin her gerçek frame'i için bir giriş vardır.
  - Her giriş, page ve ilgili process hakkında bilgiler içerir.
  - Bu nedenle, **sistemde yalnızca bir page tablosu vardır** ve fiziksel belleğin her frame'i için yalnızca bir giriş vardır.

# Inverted Page Tables

- Bu yöntemi açıklamak için, IBM RT iş istasyonu bilgisayarlarda kullanılan inverted page tablosunun basit bir örneği şekilde verilmiştir. IBM RT için, sistemdeki her sanal adres bir üçlüden oluşur: **<process-id, page-number, offset>**
- Inverted page tablosunda process-id, adres alanı tanımlayıcısı rolünü üstlenir. Tablodaki her satır **<process-id, page-number>** ikilisinden oluşur.
- Bir bellek referansı gerçekleştiğinde inverted page tablosunda bir eşleşme aranır. Örneğin tabloda i. kayıta eşleşme bulunmuşsa bu sanal adresin fiziksel adresi **<i, offset>** olarak elde edilir.



# Inverted Page Tables

- Bu şema her bir page tablosunu saklamak için gereken bellek miktarını azaltmasına rağmen, bir page referansı gerçekleştiğinde tabloyu aramak için gereken süreyi artırır.
- Inverted Page tablosu fiziksel adrese göre sıralandığından ancak aramalar sanal adreslerde gerçekleştiğinden bir eşleşme bulunmadan önce tüm tablonun aranması gerekebilir.
- Bu arama çok uzun sürer. Bu sorunu hafifletmek için aramayı bir veya en fazla birkaç page tablosu girişiyle sınırlandıran bir hash tablo kullanılabilir.
  - Hash tabloya her erişim, prosedüre bir bellek referansı ekler bu nedenle bir sanal bellek referansı, biri hash tablo girişi ve biri page tablosu için olmak üzere en az iki gerçek bellek okuması gerektirir.
  - Bir miktar performans iyileştirmesi için hash tabloya başvurmadan önce TLB kullanılabilir.



# Inverted Page Tables

- Inverted page tablolarını kullanan sistemler, paylaşılan belleği uygulamada zorluk yaşarlar.
- Standart paging'de her process'in, birden fazla sanal adresin aynı fiziksel adrese eşlenmesine izin veren kendi page tablosu vardır.
- Bu standart yöntem, Inverted page tablolarıyla kullanılamaz; her fiziksel page için yalnızca bir sanal page girişi olduğundan, bir fiziksel page'in iki (veya daha fazla) paylaşılan sanal adresi olamaz.
- Bu nedenle inverted page tablolarıyla, herhangi bir zamanda sanal bir adresin paylaşılan fiziksel adrese yalnızca bir eşlemesi gerçekleşebilir.

# Kaynaklar

- Operating System Concepts 9th edition, Silberschatz, Galvin and Gagne.
- <https://codex.cs.yale.edu/avi/os-book/OS9/slide-dir/index.html>
- <http://w3.gazi.edu.tr/~akcayol/BMOS.htm>
- <http://akademik.ube.ege.edu.tr/~gungor/courses/isletim/10.bellekler.pptx>